

第43章 ETH—Lwip 以太网通信

互联网技术对人类社会的影响不言而喻。当今大部分电子设备都能以不同的方式接入互联网(Internet)，在家庭中 PC 常见的互联网接入方式是使用路由器(Router)组建小型局域网(LAN)，利用互联网专线或者调制解调器(modem)经过电话线网络，连接到互联网服务提供商(ISP)，由互联网服务提供商把用户的局域网接入互联网。而企业或学校的局域网规模较大，常使用交换机组成局域网，经过路由以不同的方式接入到互联网中。

43.1 互联网模型

通信至少是两个设备的事，需要相互兼容的硬件和软件支持，我们称之为通信协议。以太网通信在结构比较复杂，国际标准组织将整个以太网通信结构制定了 OSI 模型，总共分层七个层，分别为应用层、表示层、会话层、传输层、网络层、数据链路层以及物理层，每个层功能不同，通信中各司其职，整个模型包括硬件和软件定义。OSI 模型是理想分层，一般的网络系统只是涉及其中几层。

TCP/IP 是互联网最基本的协议，是互联网通信使用的网络协议，由网络层的 IP 协议和传输层的 TCP 协议组成。TCP/IP 只有四个分层，分别为应用层、传输层、网络层以及网络访问层。虽然 TCP/IP 分层少了，但与 OSI 模型是不冲突的，它把 OSI 模型一些层次整合一起的，本质上可以实现相同功能。

实际上，还有一个 TCP/IP 混合模型，分为五个层，参考图 43-1，它实际与 TCP/IP 四层模型是相通的，只是把网络访问层拆成数据链路层和物理层。这种分层方法对我们学习理解更容易。

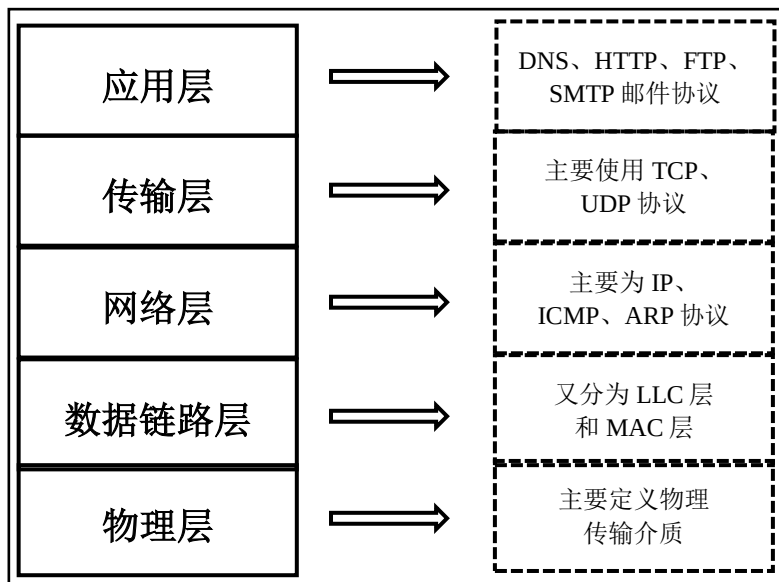


图 43-1 TCP/IP 混合参考模型

设计网络时，为了降低网络设计的复杂性，对组成网络的硬件、软件进行封装、分层，这些分层即构成了网络体系模型。在两个设备相同层之间的对话、通信约定，构成了层级协议。设备中使用的所有协议加起来统称协议栈。在这个网络模型中，每一层完成不同的

任务，都提供接口供上一层访问。而在每层的内部，可以使用不同的方式来实现接口，因而内部的改变不会影响其它层。

在 TCP/IP 混合参考模型中，数据链路层又被分为 LLC 层(逻辑链路层)和 MAC 层(媒体介质访问层)。目前，对于普通的接入网络终端的设备，LLC 层和 MAC 层是软、硬件的分界线。如 PC 的网卡主要负责实现参考模型中的 MAC 子层和物理层，在 PC 的软件系统中则有一套庞大程序实现了 LLC 层及以上的所有网络层次的协议。

由硬件实现的物理层和 MAC 子层在不同的网络形式有很大的区别，如以太网和 Wi-Fi，这是由物理传输方式决定的。但由软件实现的其它网络层次通常不会有太大区别，在 PC 上也许能实现完整的功能，一般支持所有协议，而在嵌入式领域则按需要进行裁剪。

43.2 以太网

以太网(Ethernet)是互联网技术的一种，由于它是在组网技术中占的比例最高，很多人直接把以太网理解为互联网。

以太网是指遵守 IEEE 802.3 标准组成的局域网，由 IEEE 802.3 标准规定的主要是位于参考模型的物理层(PHY)和数据链路层中的介质访问控制子层(MAC)。在家庭、企业和学校所组建的 PC 局域网形式一般也是以太网，其标志是使用水晶头网线来连接(当然还有其它形式)。IEEE 还有其它局域网标准，如 IEEE 802.11 是无线局域网，俗称 Wi-Fi。IEEE 802.15 是个人域网，即蓝牙技术，其中的 802.15.4 标准则是 ZigBee 技术。

现阶段，工业控制、环境监测、智能家居的嵌入式设备产生了接入互联网的需求，利用以太网技术，嵌入式设备可以非常容易地接入到现有的计算机网络中。

43.2.1 PHY 层

在物理层，由 IEEE 802.3 标准规定了以太网使用的传输介质、传输速度、数据编码方式和冲突检测机制，物理层一般是通过一个 PHY 芯片实现其功能的。

1. 传输介质

传输介质包括同轴电缆、双绞线(水晶头网线是一种双绞线)、光纤。根据不同的传输速度和距离要求，基于这三类介质的信号线又衍生出很多不同的种类。最常用的是“五类线”适用于 100BASE-T 和 10BASE-T 的网络，它们的网络速率分别为 100Mbps 和 10Mbps。

2. 编码

为了让接收方在没有外部时钟参考的情况也能确定每一位的起始、结束和中间位置，在传输信号时不直接采用二进制编码。在 10BASE-T 的传输方式中采用曼彻斯特编码，在 100BASE-T 中则采用 4B/5B 编码。

曼彻斯特编码把每一个二进制位的周期分为两个间隔，在表示“1”时，以前半个周期为高电平，后半周期为低电平。表示“0”时则相反，见图 43-2

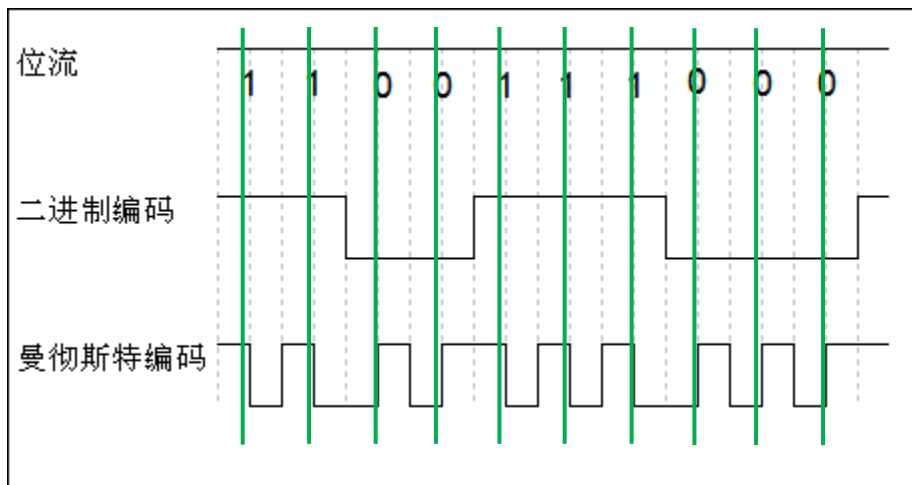


图 43-2 曼彻斯特编码

采用曼彻斯特码在每个位周期都有电压变化，便于同步。但这样的编码方式效率太低，只有 50%。

在 100BASE-T 采用的 4B/5B 编码是把待发送数据位流的每 4 位分为一组，以特定的 5 位编码来表示，这些特定的 5 位编码能使数据流有足够多的跳变，达到同步的目的，而且效率也从曼彻斯特编码的 50%提高到了 80%。

3. CSMA/CD 冲突检测

早期的以太网大多是多个节点连接到同一条网络总线上(总线型网络)，存在信道竞争问题，因而每个连接到以太网上的节点都必须具备冲突检测功能。以太网具备 CSMA/CD 冲突检测机制，如果多个节点同时利用同一条总线发送数据，则会产生冲突，总线上的节点可通过接收到的信号与原始发送的信号的比较检测是否存在冲突，若存在冲突则停止发送数据，随机等待一段时间再重传。

现在大多数局域网组建的时候很少采用总线型网络，大多是一个设备接入到一个独立的路由或交换机接口，组成星型网络，不会产生冲突。但为了兼容，新出的产品还是带有冲突检测机制。

43.2.2 MAC 子层

1. MAC 的功能

MAC 子层是属于数据链路层的下半部分，它主要负责与物理层进行数据交接，如是否可以发送数据，发送的数据是否正确，对数据流进行控制等。它自动对来自上层的数据包加上一些控制信号，交给物理层。接收方得到正常数据时，自动去除 MAC 控制信号，把该数据包交给上层。

2. MAC 数据包

IEEE 对以太网上传输的数据包格式也进行了统一规定，见图 43-3。该数据包被称为 MAC 数据包。

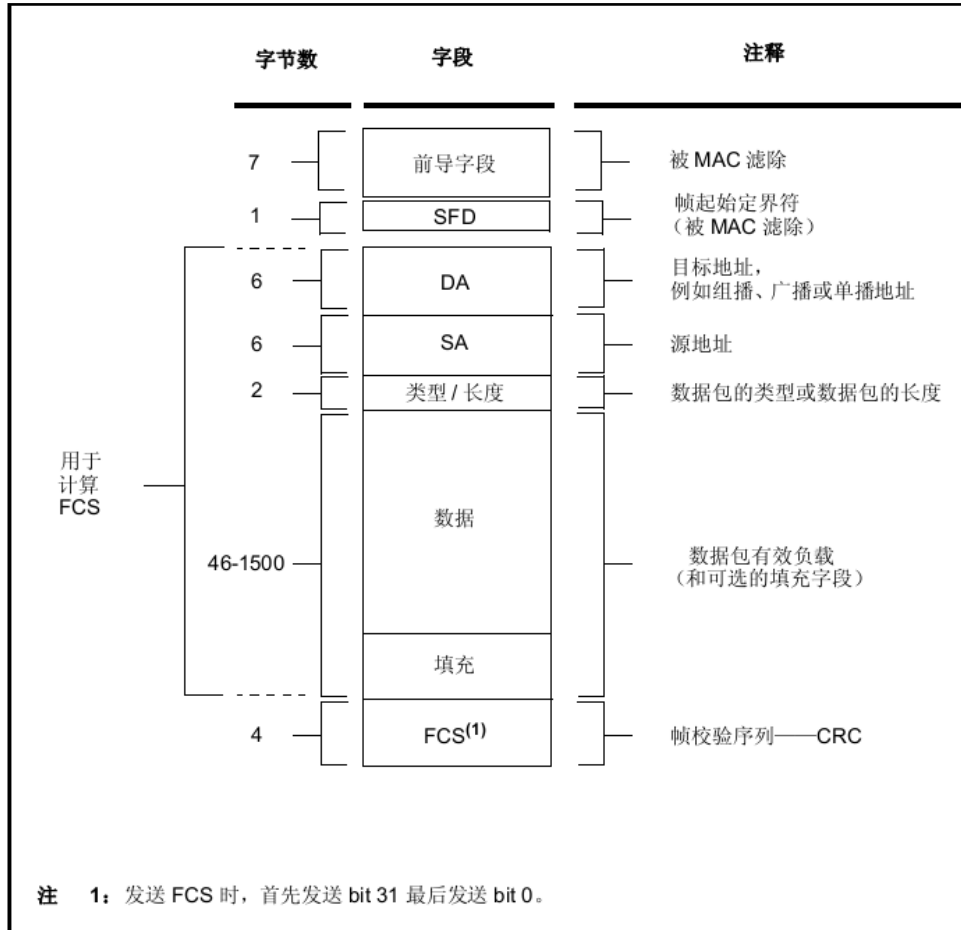


图 43-3 MAC 数据包格式

MAC 数据包由前导字段、帧起始定界符、目标地址、源地址、数据包类型、数据域、填充域、校验和域组成。

- ❑ 前导字段，也称报头，这是一段方波，用于使收发节点的时钟同步。内容为连续 7 个字节的 0x55。字段和帧起始定界符在 MAC 收到数据包后会自动过滤掉。
- ❑ 帧起始定界符(SFD): 用于区分前导段与数据段的，内容为 0xD5。
- ❑ MAC 地址: MAC 地址由 48 位数字组成，它是网卡的物理地址，在以太网传输的最底层，就是根据 MAC 地址来收发数据的。部分 MAC 地址用于广播和多播，在同一个网络里不能有两个相同的 MAC 地址。PC 的网卡在出厂时已经设置好了 MAC 地址，但也可以通过一些软件来进行修改，在嵌入式的以太网控制器中可由程序进行配置。数据包中的 DA 是目标地址，SA 是源地址。
- ❑ 数据包类型: 本区域可以用来描述本 MAC 数据包是属于 TCP/IP 协议层的 IP 包、ARP 包还是 SNMP 包，也可以用来描述本 MAC 数据包数据段的长度。如果该值被设置大

于 0x0600，不用于长度描述，而是用于类型描述功能，表示与以太网帧相关的 MAC 客户端协议的种类。

- ❑ 数据段：数据段是 MAC 包的核心内容，它包含的数据来自 MAC 的上层。其长度可以从 0~1500 字节间变化。
- ❑ 填充域：由于协议要求整个 MAC 数据包的长度至少为 64 字节(接收到的数据包如果少于 64 字节会被认为发生冲突，数据包被自动丢弃)，当数据段的字节少于 46 字节时，在填充域会自动填上无效数据，以使数据包符合长度要求。
- ❑ 校验和域：MAC 数据包的尾部是校验和域，它保存了 CRC 校验序列，用于检错。

以上是标准的 MAC 数据包，IEEE 802.3 同时还规定了扩展的 MAC 数据包，它是在标准的 MAC 数据包的 SA 和数据包类型之间添加 4 个字节的 QTag 前缀字段，用于获取标志的 MAC 帧。前 2 个字节固定为 0x8100，用于识别 QTag 前缀的存在；后两个字节内容分别为 3 个位的用户优先级、1 个位的标准格式指示符(CFI)和一个 12 位的 VLAN 标识符。

43.3 TCP/IP 协议栈

标准 TCP/IP 协议是用于计算机通信的一组协议，通常称为 TCP/IP 协议栈，通俗讲就是符合以太网通信要求的代码集合，一般要求它可以实现图 43-1 中每个层对应的协议，比如应用层的 HTTP、FTP、DNS、SMTP 协议，传输层的 TCP、UDP 协议、网络层的 IP、ICMP 协议等等。关于 TCP/IP 协议详细内容推荐阅读《TCP-IP 详解》和《用 TCP/IP 进行网际互连》理解。

Windows 操作系统、UNIX 类操作系统都有自己的一套方法来实现 TCP/IP 通信协议，它们都提供非常完整的 TCP/IP 协议。对于一般的嵌入式设备，受制于硬件条件没办法支持使用在 Window 或 UNIX 类操作系统的运行的 TCP/IP 协议栈，一般只能使用简化版本的 TCP/IP 协议栈，目前开源的适合嵌入式的有 uIP、TinyTCP、uC/TCP-IP、LwIP 等等。其中 LwIP 是目前在嵌入式网络领域被讨论和使用广泛的协议栈。本章内容其中一个目的就是移植 LwIP 到开发板上运行。

43.3.1 为什么需要协议栈

物理层主要定义物理介质性质，MAC 子层负责与物理层进行数据交接，这两部分是与硬件紧密联系的，就嵌入式控制芯片来说，很多都内部集成了 MAC 控制器，完成 MAC 子层功能，所以依靠这部分功能是可以实现两个设备数据交换，而时间传输的数据就是 MAC 数据包，发送端封装好数据包，接收端则解封数据包得到可用数据，这样的模型与使用 USART 控制器实现数据传输是非常类似的。但如果将以太网运用在如此基础的功能上，完全是大材小用，因为以太网具有传输速度快、可传输距离远、支持星型拓扑设备连接等等强大功能。功能强大的东西一般都会用高级的应用，这也是设计者的初衷。

使用以太网接口的目的就是为了方便与其它设备互联，如果所有设备都约定使用一种互联方式，在软件上加一些层次来封装，这样不同系统、不同的设备通讯就变得相对容易了。而且只要新加入的设备也使用同一种方式，就可以直接与之前存在于网络上的其它设备通讯。这就是为什么产生了在 MAC 之上的其它层次的网络协议及为什么要使用协议栈

的原因。又由于在各种协议栈中 TCP/IP 协议栈得到了最广泛使用，所有接入互联网的设备都遵守 TCP/IP 协议。所以，想方便地与其它设备互联通信，需要提供对 TCP/IP 协议的支持。

43.3.2 各网络层的功能

用以太网和 Wi-Fi 作例子，它们的 MAC 子层和物理层有较大的区别，但在 MAC 之上的 LLC 层、网络层、传输层和应用层的协议，是基本相同的，这几层协议由软件实现，并对各层进行封装。根据 TCP/IP 协议，各层的要实现的功能如下：

LLC 层：处理传输错误；调节数据流，协调收发数据双方速度，防止发送方发送得太快而接收方丢失数据。主要使用数据链路协议。

网络层：本层也被称为 IP 层。LLC 层负责把数据从线的一端传输到另一端，但很多时候不同的设备位于不同的网络中(并不是简单的网线的两头)。此时就需要网络层来解决子网路由拓扑问题、路径选择问题。在这一层主要有 IP 协议、ICMP 协议。

传输层：由网络层处理好了网络传输的路径问题后，端到端的路径就建立起来了。传输层就负责处理端到端的通讯。在这一层中主要有 TCP、UDP 协议

应用层：经过前面三层的处理，通讯完全建立。应用层可以通过调用传输层的接口来编写特定的应用程序。而 TCP/IP 协议一般也会包含一些简单的应用程序如 Telnet 远程登录、FTP 文件传输、SMTP 邮件传输协议。

实际上，在发送数据时，经过网络协议栈的每一层，都会给来自上层的数据添加上一个数据包的头，再传递给下一层。在接收方收到数据时，一层层地把所在层的数据包的头去掉，向上层递交数据，参考图 43-4。

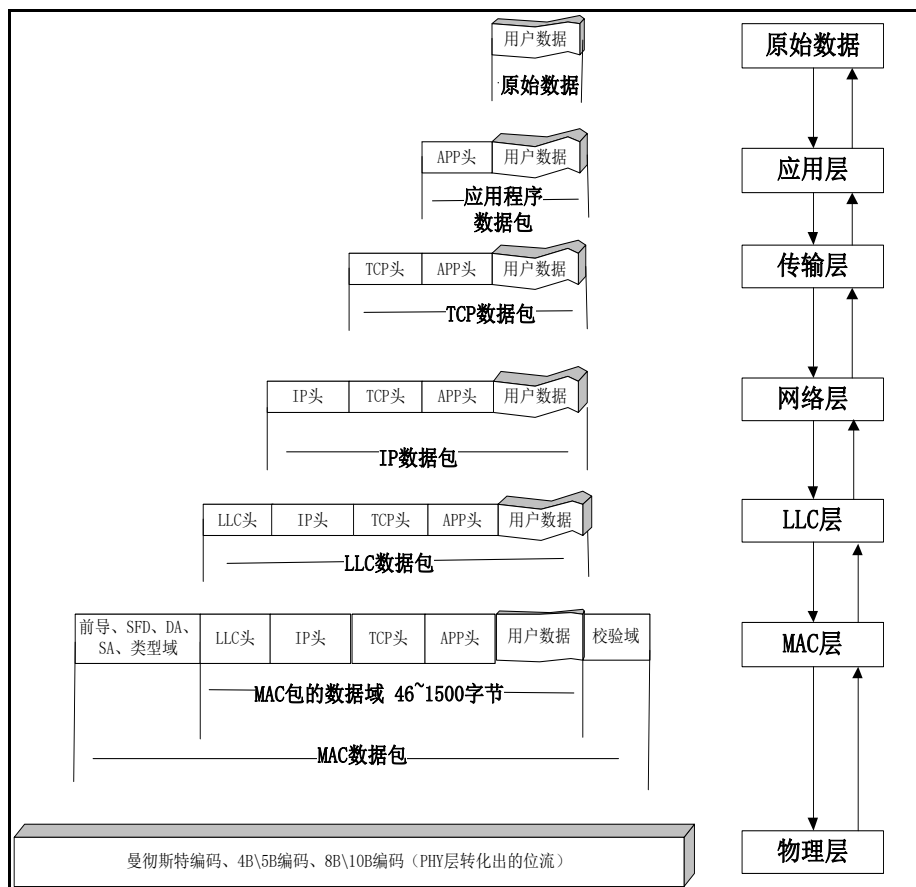


图 43-4 数据经过每一层的封装和还原

43.4 以太网外设(ETH)

STM32F4xx 系列控制器内部集成了一个以太网外设，它实际是一个通过 DMA 控制器进行介质访问控制(MAC)，它的功能就是实现 MAC 层的任务。借助以太网外设，STM32F4xx 控制器可以通过 ETH 外设按照 IEEE 802.3-2002 标准发送和接收 MAC 数据包。ETH 内部自带专用的 DMA 控制器用于 MAC，ETH 支持两个工业标准接口介质独立接口(MII)和简化介质独立接口(RMII)用于与外部 PHY 芯片连接。MII 和 RMII 接口用于 MAC 数据包传输，ETH 还集成了站管理接口(SMI)接口专门用于与外部 PHY 通信，用于访问 PHY 芯片寄存器。

物理层定义了以太网使用的传输介质、传输速度、数据编码方式和冲突检测机制，PHY 芯片是物理层功能实现的实体，生活中常用水晶头网线+水晶头插座+PHY 组合构成了物理层。

ETH 有专用的 DMA 控制器，它通过 AHB 主从接口与内核和存储器相连，AHB 主接口用于控制数据传输，而 AHB 从接口用于访问“控制与状态寄存器”(CSR)空间。在进行数据发送是，先将数据有存储器以 DMA 传输到发送 TX FIFO 进行缓冲，然后由 MAC 内核发送；接收数据时，RX FIFO 先接收以太网数据帧，再由 DMA 传输至存储器。ETH 系统功能框图见图 43-5。

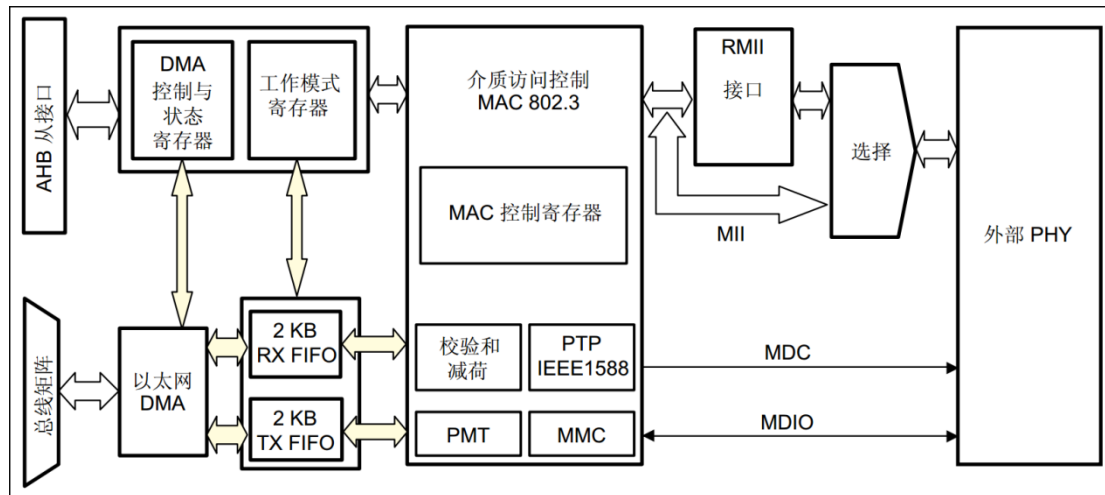


图 43-5 ETH 功能框图

43.4.1 SMI 接口

SMI 是 MAC 内核访问 PHY 寄存器标志接口，它由两根线组成，数据线 MDIO 和时钟线 MDC。SMI 支持访问 32 个 PHY，这在设备需要多个网口时非常有用，不过一般设备都只使用一个 PHY。PHY 芯片内部一般都有 32 个 16 位的寄存器，用于配置 PHY 芯片属性、工作环境、状态指示等等，当然很多 PHY 芯片并没有使用到所有寄存器位。MAC 内核就是通过 SMI 向 PHY 的寄存器写入数据或从 PHY 寄存器读取 PHY 状态，一次只能对一个 PHY 的其中一个寄存器进行访问。SMI 最大通信频率为 2.5MHz，通过控制以太网 MAC MII 地址寄存器 (ETH_MACMIAR) 的 CR 位可选择时钟频率。

1. SMI 帧格式

SMI 是通过数据帧方式与 PHY 通信的，帧格式如表 43-1，数据位传输顺序从左到右。

表 43-1 SMI 帧格式

	管理帧字段							
	报头(32bit)	起始	操作	PADDR	RADDR	TA	数据(16bit)	空闲
读取	111...111	01	10	ppppp	rrrrr	Z0	ddd...ddd	Z
写入	111...111	01	01	ppppp	rrrrr	10	ddd...ddd	Z

PADDR 用于指定 PHY 地址，每个 PHY 都有一个地址，一般由 PHY 硬件设计决定，所以是固定不变的。RADDR 用于指定 PHY 寄存器地址。TA 为状态转换域，若为读操作，MAC 输出两个位高阻态，而 PHY 芯片则在第一位时输出高阻态，第二位时输出“0”。若为写操作，MAC 输出“10”，PHY 芯片则输出高阻态。数据段有 16 位，对应 PHY 寄存器每个位，先发送或接收到的位对应以太网 MAC MII 数据寄存器(ETH_MACMIIDR)寄存器的位 15。

2. SMI 读写操作

当以太网 MAC MII 地址寄存器 (ETH_MACMIAR) 的写入位和繁忙位被置 1 时, SMI 将向指定的 PHY 芯片指定寄存器写入 ETH_MACMIIDR 中的数据。写操作时序见图 43-6。

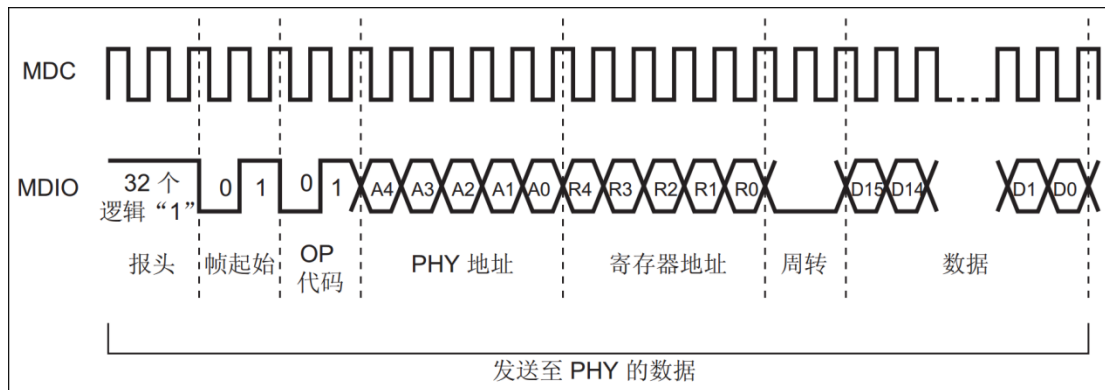


图 43-6 SMI 写操作

当以太网 MAC MII 地址寄存器 (ETH_MACMIAR) 的写入位为 0 并且繁忙位被置 1 时, SMI 将从向指定的 PHY 芯片指定寄存器读取数据到 ETH_MACMIIDR 内。读操作时序见图 43-7。

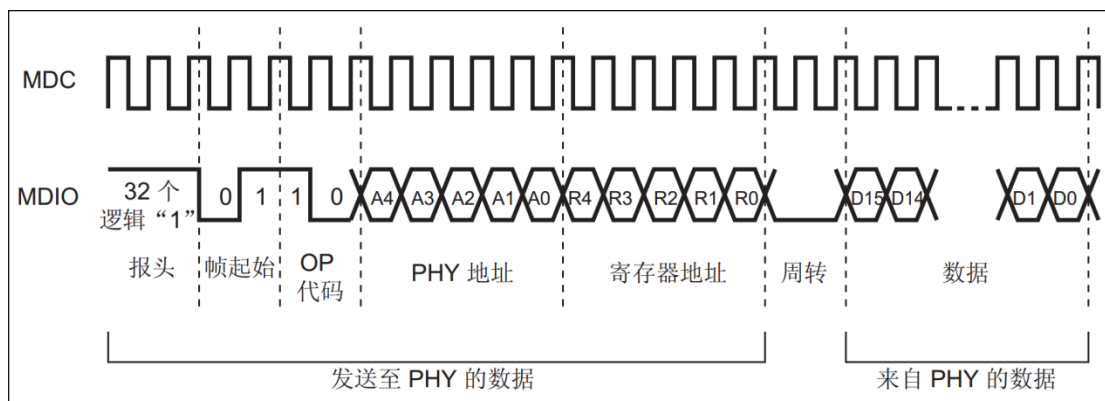


图 43-7 SMI 读操作

43.4.2 MII 和 RMII 接口

介质独立接口(MII)用于连接 MAC 控制器和 PHY 芯片, 提供数据传输路径。RMII 接口是 MII 接口的简化版本, MII 需要 16 根通信线, RMII 只需 7 根通信, 在功能上是相同的。图 43-8 为 MII 接口连接示意图, 图 43-9 为 RMII 接口连接示意图。

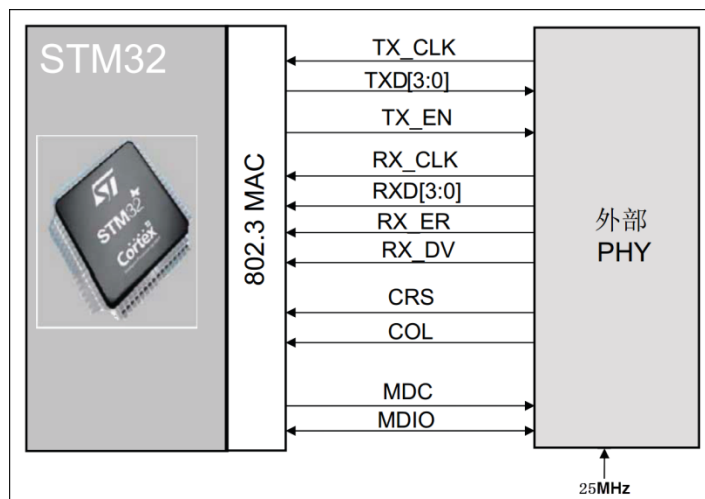


图 43-8 MII 接口连接

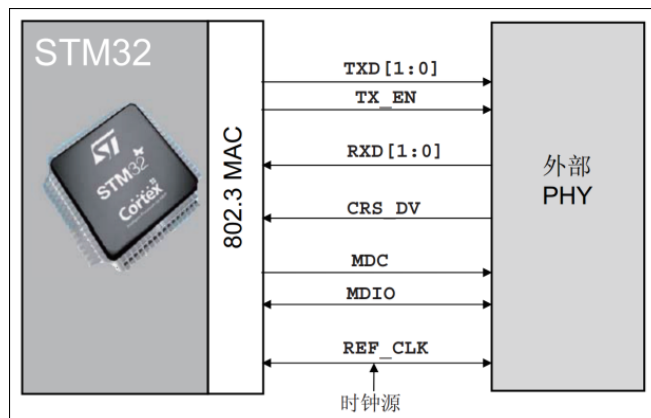


图 43-9 RMII 接口连接

- ❑ TX_CLK: 数据发送时钟线。标称速率为 10Mbit/s 时为 2.5MHz; 速率为 100Mbit/s 时为 25MHz。RMII 接口没有该线。
- ❑ RX_CLK: 数据接收时钟线。标称速率为 10Mbit/s 时为 2.5MHz; 速率为 100Mbit/s 时为 25MHz。RMII 接口没有该线。
- ❑ TX_EN: 数据发送使能。在整个数据发送过程保存有效电平。
- ❑ TXD[3:0]或 TXD[1:0]: 数据发送数据线。对于 MII 有 4 位, RMII 只有 2 位。只有在 TX_EN 处于有效电平数据线才有效。
- ❑ CRS: 载波侦听信号, 由 PHY 芯片负责驱动, 当发送或接收介质处于非空闲状态时使能该信号。在全双工模式该信号线无效。
- ❑ COL: 冲突检测信号, 由 PHY 芯片负责驱动, 检测到介质上存在冲突后该线被使能, 并且保持至冲突解除。在全双工模式该信号线无效。
- ❑ RXD[3:0]或 RXD[1:0]: 数据接收数据线, 由 PHY 芯片负责驱动。对于 MII 有 4 位, RMII 只有 2 位。在 MII 模式, 当 RX_DV 禁止、RX_ER 使能时, 特定的 RXD[3:0]值用于传输来自 PHY 的特定信息。

- ❑ RX_DV: 接收数据有效信号, 功能类似 TX_EN, 只不过用于数据接收, 由 PHY 芯片负责驱动。对于 RMII 接口, 是把 CRS 和 RX_DV 整合成 CRS_DV 信号线, 当介质处于不同状态时会自切换该信号状态。
- ❑ RX_ER: 接收错误信号线, 由 PHY 驱动, 向 MAC 控制器报告在帧某处检测到错误。
- ❑ REF_CLK: 仅用于 RMII 接口, 由外部时钟源提供 50MHz 参考时钟。

因为要达到 100Mbit/s 传输速度, MII 和 RMII 数据线数量不同, 使用 MII 和 RMII 时钟线的设计是完全不同的。对于 MII 接口, 一般是外部为 PHY 提供 25MHz 时钟源, 再由 PHY 提供 TX_CLK 和 RX_CLK 时钟。对于 RMII 接口, 一般需要外部直接提供 50MHz 时钟源, 同时接入 MAC 和 PHY。

开发板板载的 PHY 芯片型号为 LAN8720A, 该芯片只支持 RMII 接口, 电路设计时参考图 43-9。

ETH 相关硬件在 STM32F4xx 控制器分布参考表 43-2。

表 43-2 ETH 复用引脚

ETH(AF11)		GPIO
MII	MII_TX_CLK	PC3
	MII_TXD0	PB12/PG13
	MII_TXD1	PB13/PG14
	MII_TXD2	PC2
	MII_TXD3	PB8/PE2
	MII_TX_EN	PB11/PG11
	MII_RX_CLK	PA1
	MII_RXD0	PC4
	MII_RXD1	PC5
	MII_RXD2	PB0
	MII_RXD3	PB1
	MII_RX_ER	PB10
	MII_RX_DV	PA7
	MII_CRD	PA0
	MII_COL	PA3
RMII	RMII_TXD0	PB12/PG13
	RMII_TXD1	PB13/PG14
	RMII_TX_EN	PG11
	RMII_RXD0	PC4
	RMII_RXD1	PC5
	RMII_CRD_DV	PA7
	RMII_REF_CLK	PA1
SMI	MDIO	PA2
	MDC	PC1
其他	PPS_OUT	PB5/PG8

其中, PPS_OUT 是 IEEE 1588 定义的一个时钟同步机制。

43.4.3 MAC 数据包发送和接收

ETH 外设负责 MAC 数据包发送和接收。利用 DMA 从系统寄存器得到数据包数据内容, ETH 外设自动填充完成 MAC 数据包封装, 然后通过 PHY 发送出去。在检测到有 MAC 数据包需要接收时, ETH 外设控制数据接收, 并解封 MAC 数据包得到解封后数据通过 DMA 传输到系统寄存器内。

1. MAC 数据包发送

MAC 数据帧发送全部由 DMA 控制, 从系统存储器读取的以太网帧由 DMA 推入 FIFO, 然后将帧弹出并传输到 MAC 内核。帧传输结束后, 从 MAC 内核获取发送状态并传回 DMA。在检测到 SOF(Start Of Frame)时, MAC 接收数据并开始 MII 发送。在 EOF(End Of Frame)传输到 MAC 内核后, 内核将完成正常的发送, 然后将发送状态返回给 DMA。如果在发送过程中发送常规冲突, MAC 内核将使发送状态有效, 然后接受并丢弃所有后续数据, 直至收到下一 SOF。检测到来自 MAC 的重试请求时, 应从 SOF 重新发送同一帧。如果发送期间未连续提供数据, MAC 将发出下溢状态。在帧的正常传输期间, 如果 MAC 在未获得前一帧的 EOF 的情况下接收到 SOF, 则将忽略该 SOF 并将新的帧视为前一帧的延续。

MAC 控制 MAC 数据包的发送操作, 它会自动生成前导字段和 SFD 以及发送帧状态返回给 DMA, 在半双工模式下自动生成阻塞信号, 控制 jabber(MAC 看门狗)定时器用于在传输字节超过 2048 字节时切断数据包发送。在半双工模式下, MAC 使用延迟机制进行流量控制, 程序通过将 ETH_MACFCR 寄存器的 BPA 位置 1 来请求流量控制。MAC 包含符合 IEEE 1588 的时间戳快照逻辑。MAC 数据包发送时序参考图 43-10。

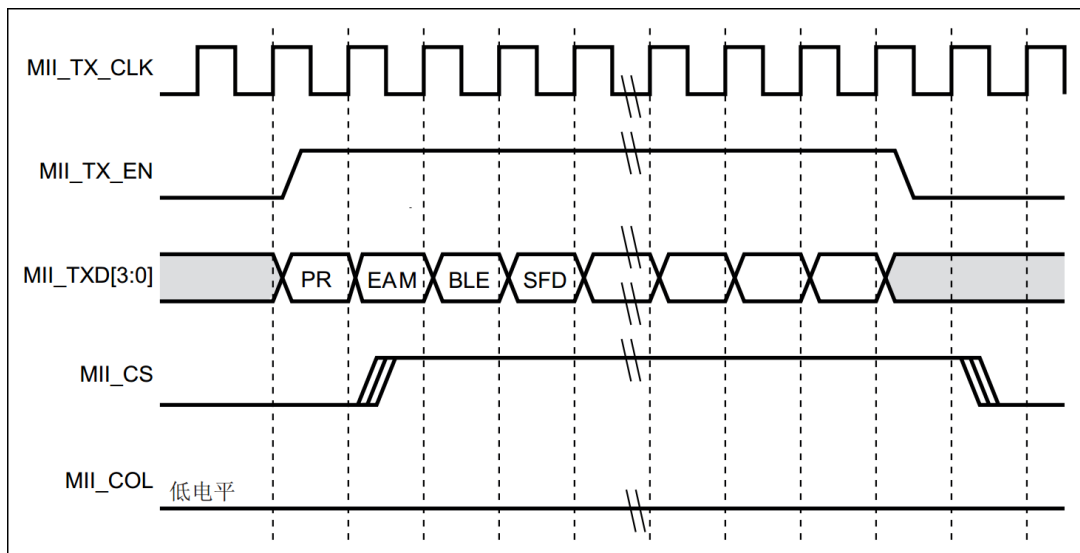


图 43-10 MAC 数据包发送时序(无冲突)

2. MAC 数据包接收

MAC 接收到的数据包填充 RX FIFO, 达到 FIFO 设定阈值后请求 DMA 传输。在默认直通模式下, 当 FIFO 接收到 64 个字节(使用 ETH_DMAOMR 寄存器中的 RTC 位配置)或

完整的数据包时，数据将弹出，其可用性将通知给 DMA。DMA 向 AHB 接口发起传输后，数据传输将从 FIFO 持续进行，直到传输完整整个数据包。完成 EOF 帧的传输后，状态字将弹出并发送到 DMA 控制器。在 Rx FIFO 存储转发模式(通过 ETH_DMAOMR 寄存器中的 RSF 位配置)下，仅在帧完全写入 Rx FIFO 后才可读出帧。

当 MAC 在 MII 上检测到 SFD 时，将启动接收操作。MAC 内核将去除报头和 SFD，然后再继续处理帧。检查报头字段以进行过滤，FCS 字段用于验证帧的 CRC 如果帧未通过地址滤波器，则在内核中丢弃该帧。MAC 数据包接收时序参考图 43-11。

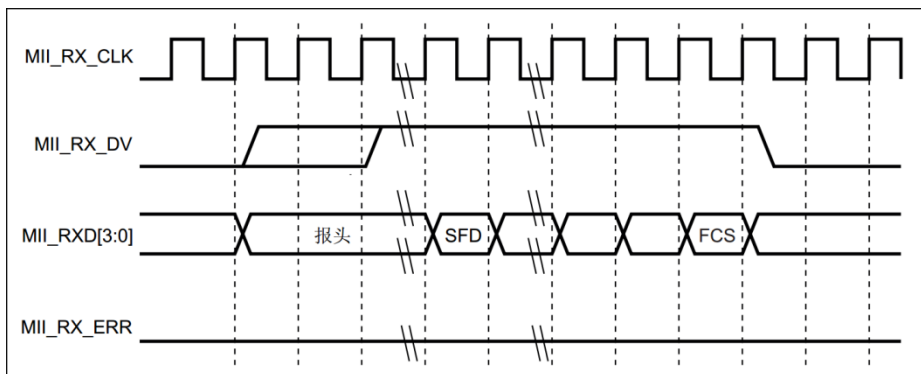


图 43-11 MAC 数据包接收时序(无错误)

43.4.4 MAC 过滤

MAC 过滤功能可以选择性的过滤设定目标地址或源地址的 MAC 帧。它将检查所有接收到的数据帧的目标地址和源地址，根据过滤选择设定情况，检测后报告过滤状态。针对目标地址过滤可以有三种，分别是单播、多播和广播目标地址过滤；针对源地址过滤就只有单播源地址过滤。

单播目标地址过滤是将接收的相应 DA 字段与预设的以太网 MAC 地址寄存器内容比较，最高可预设 4 个过滤 MAC 地址。多播目标地址过滤是根据帧过滤寄存器中的 HM 位执行对多播地址的过滤，是对 MAC 地址寄存器进行比较来实现的。单播和多播目标地址过滤都还支持 Hash 过滤模式。广播目标地址过滤通过将帧过滤寄存器的 BFD 位置 1 使能，这使得 MAC 丢弃所有广播帧。

单播源地址过滤是将接收的 SA 字段与 SA 寄存器内容进行比较过滤。

MAC 过滤还具备反向过滤操作功能，即让过滤结构求补集。

43.5 PHY: LAN8720A

LAN8720A 是 SMSC 公司(已被 Microchip 公司收购)设计的一个体积小、功耗低、全能型 10/100Mbps 的以太网物理层收发器。它是针对消费类电子和企业应用而设计的。LAN8720A 总共只有 24Pin，仅支持 RMII 接口。由它组成的网络结构见图 43-12。

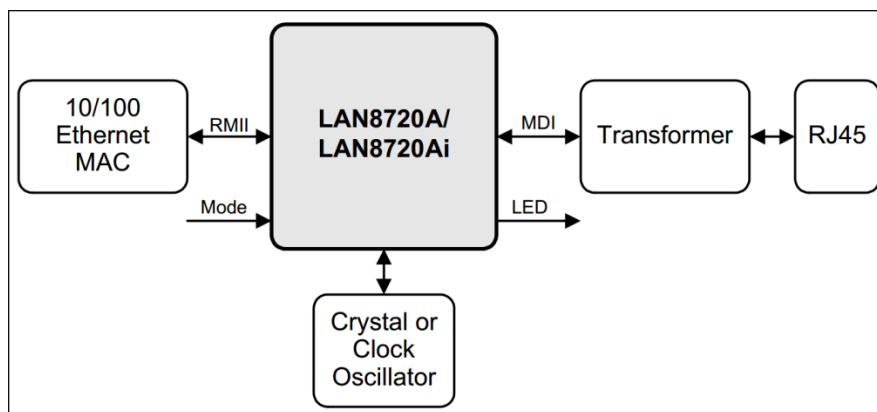


图 43-12 由 LAN8720A 组成的网络系统结构

LAN8720A 通过 RMIi 与 MAC 连接。RJ45 是网络插座，在与 LAN8720A 连接之间还需要一个变压器，所以一般使用带电压转换和 LED 指示灯的 HY911105A 型号的插座。一般来说，必须为使用 RMIi 接口的 PHY 提供 50MHz 的时钟源输入到 REF_CLK 引脚，不过 LAN8720A 内部集成 PLL，可以将 25MHz 的时钟源倍频到 50MHz 并在指定引脚输出该时钟，所以我们可以直接使其与 REF_CLK 连接达到提供 50MHz 时钟的效果。

LAN8720A 内部系统结构见图 43-13。

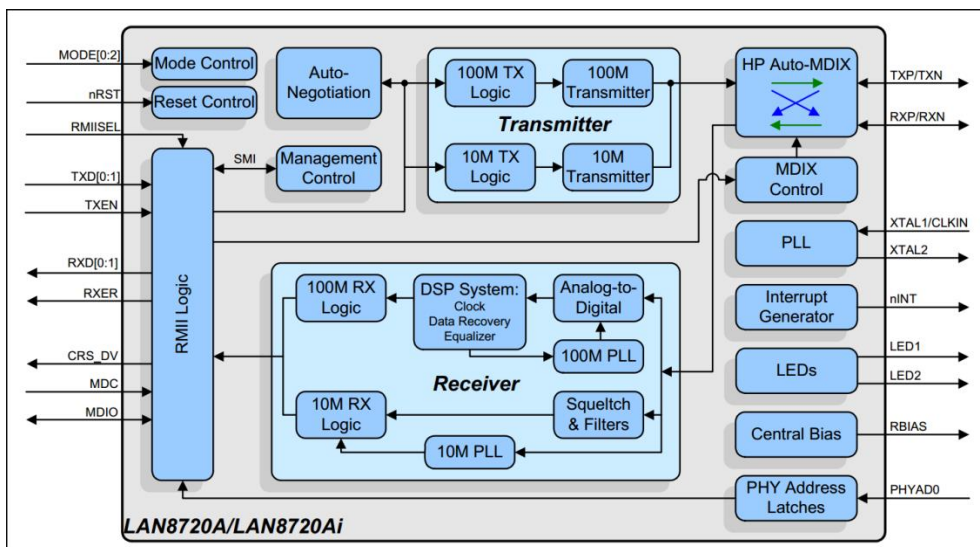


图 43-13 LAN8720A 内部系统结构

LAN8720A 有各个不同功能模块组成，最重要的要数接收控制器和发送控制器，其它的基本上都是与外部引脚挂钩，实现信号传输。部分引脚是具有双重功能的，比如 PHYAD0 与 RXER 引脚是共用的，在系统上电后 LAN8720A 会马上读取这部分共用引脚的电平，以确定系统的状态并保存在相关寄存器内，之后则自动转入作为另一功能引脚。

PHYAD[0]引脚用于配置 SMI 通信的 LAN8720A 地址，在芯片内部该引脚已经自带下拉电阻，默认为 0(即使外部悬空不接)，在系统上电时会检测该引脚获取到 LAN8720A 的地址为 0 或者 1，并保存在特殊模式寄存器(R18)的 PHYAD 位中，该寄存器的 PHYAD 有 5 个位，在需要超过 2 个 LAN8720A 时可以通过软件设置不同 SMI 通信地址。PHYAD[0]是与 RXER 引脚共用。

MODE[2:0]引脚用于选择 LAN8720A 网络通信速率和工作模式，可选 10Mbps 或 100Mbps 通信速度，半双工或全双工工作模式，另外 LAN8720A 支持 HP Auto-MDIX 自动翻转功能，即可自动识别直连或交叉网线并自适应。一般将 MODE 引脚都设置为 1，可以让 LAN8720A 启动自适应功能，它会自动寻找最优工作方式。MODE[0]与 RXD0 引脚共用、MODE[1]与 RXD1 引脚共用、MODE[2]与 CRS_DV 引脚共用。

nINT/REFCLKO 引脚用于 RMII 接口中 REF_CLK 信号线，当 nINTSEL 引脚为低电平时，它也可以被设置成 50MHz 时钟输出，这样可以直接与 STM32F4xx 的 REF_CLK 引脚连接为其提供 50MHz 时钟源，这种模式要求为 XTAL1 与 XTAL2 之间或为 XTAL1/CLKIN 提供 25MHz 时钟，由 LAN8720A 内部 PLL 电路倍频得到 50MHz 时钟，此时 nIN/REFCLKO 引脚的中断功能不可用，用于 50MHz 时钟输出。当 nINTSEL 引脚为高电平时，LAN8720A 被设置为时钟输入，即外部时钟源直接提供 50MHz 时钟接入 STM32F4xx 的 REF_CLK 引脚和 LAN8720A 的 XTAL1/CLKIN 引脚，此时 nINT/REFCLKO 可用于中断功能。nINTSEL 与 LED2 引脚共用，一般使用下拉

REGOFF 引脚用于配置内部+1.2V 电压源，LAN8720A 内部需要+1.2V 电压，可以通过 VDDCR 引脚输入+1.2V 电压提供，也可以直接利用 LAN8720A 内部+1.2V 稳压器提供。当 REGOFF 引脚为低电平时选择内部+1.2V 稳压器。REGOFF 与 LED1 引脚共用。

SMI 支持寻址 32 个寄存器，LAN8720A 只用到其中 14 个，参考表 43-3。

表 43-3 LAN8720A 寄存器列表

序号	寄存器名称	分组
0	Basic Control Register	Basic
1	Basic Status Register	Basic
2	PHY Identifier 1	Extended
3	PHY Identifier 2	Extended
4	Auto-Negotiation Advertisement Register	Extended
5	Auto-Negotiation Link Partner Ability Register	Extended
6	Auto-Negotiation Expansion Register	Extended
17	Mode Control/Status Register	Vendor-specific
18	Special Modes	Vendor-specific
26	Symbol Error Counter Register	Vendor-specific
27	Control / Status Indication Register	Vendor-specific
29	Interrupt Source Register	Vendor-specific
30	Interrupt Mask Register	Vendor-specific
31	PHY Special Control/Status Register	Vendor-specific

序号与 SMI 数据帧中的 RADDR 是对应的，这在编写驱动时非常重要，本文将它们标记为 R0~R31。寄存器可规划为三个组：Basic、Extended 和 Vendor-specific。Basic 是 IEEE 802.3 要求的，R0 是基本控制寄存器，其位 15 为 Soft Reset 位，向该位写 1 启动 LAN8720A 软件复位，还包括速度、自适应、低功耗等功能设置。R1 是基本状态寄存器。Extended 是扩展寄存器，包括 LAN8720A 的 ID 号、制造商、版本号等信息。Vendor-specific 是供应商自定义寄存器，R31 是特殊控制/状态寄存器，指示速度类型和自适应功能。

43.6 LwIP：轻型 TCP/IP 协议栈

LwIP 是 Light Weight Internet Protocol 的缩写，是由瑞士计算机科学院 Adam Dunkels 等开发的适用于嵌入式领域的开源轻量级 TCP/IP 协议栈。它可以移植到含有操作系统的平台中，也可以在无操作系统的平台下运行。由于它开源、占用的 RAM 和 ROM 比较少、支持较为完整的 TCP/IP 协议、且十分便于裁剪、调试，被广泛应用在中低端的 32 位控制器平台。可以访问网站：<http://savannah.nongnu.org/projects/lwip/> 获取更多 LwIP 信息。

目前，LwIP 最新更新到 1.4.1 版本，我们在上述网站可找到相应的 LwIP 源码下载通道。我们下载两个压缩包：lwip-1.4.1.zip 和 contrib-1.4.1.zip，lwip-1.4.1.zip 包括了 LwIP 的实现代码，contrib-1.4.1.zip 包含了不同平台移植 LwIP 的驱动代码和使用 LwIP 实现的一些应用实例测试。

但是，遗憾的是 contrib-1.4.1.zip 并没有为 STM32 平台提供实例，这对于初学者想要移植 LwIP 来说难度还是非常大的。ST 公司也是认识到 LwIP 在嵌入式领域的重要性，所以他们针对 LwIP 应用开发了测试平台，其中有一个是在 STM32F4x7 系列控制器运行的（文件编号为：[STSW-STM32070](#)）。为减少移植工作量，我们选择使用 ST 官方例程相关文件，特别是 ETH 底层驱动部分函数，这样我们也可以花更多精力在理解代码实现方法上。

本章的一个重点内容就是介绍 LwIP 移植至我们的开发平台，详细的操作步骤参考下文介绍。

43.7 ETH 初始化结构体详解

一般情况下，标准库都会为外设建立一个外设对应的文件存放外设相关库函数的实现，比如 stm32f4xx_adc.c、stm32f4xx_can.c 等等，然而标准库并没有为 ETH 外设建立相关的文件，这样我们根本没有标准库函数可以使用，究其原因 ETH 驱动函数与 PHY 芯片连续较为紧密，很难使用一套通用的代码实现兼容。难道要我们自己写寄存器实现？实际情况还没有这么糟糕，正如上文所说的 ST 官方有提供 LwIP 方面的测试平台，特别是基于 STM32F4x7 控制器的测试平台是非常合适我们参考的。我们在解压 stsw-stm32070.rar 压缩包之后，在其文件目录

(...\STM32F4x7_ETH_LwIP_V1.1.1\Libraries\STM32F4x7_ETH_Driver\)下可找到

stm32f4x7_eth.c、stm32f4x7_eth.h 和 stm32f4x7_eth_conf_template.h 三个文件，其中的 stm32f4x7_eth.c 和 stm32f4x7_eth.h 就是类似 stm32f4xx_adc.c 是关于 ETH 外设的驱动，我们在以太网通信实现实验中会使用到这三个文件，stm32f4x7_eth.c 和 stm32f4x7_eth.h 两个文件内容不用修改。

stm32f4x7_eth.h 有定义了一个 ETH 外设初始化结构体 ETH_InitTypeDef，理解结构体成员可以帮助我们使用 ETH 功能。初始化结构体成员用于设置 ETH 工作环境参数，并由 ETH 相应初始化配置函数或功能函数调用，这些设定参数将会设置 ETH 相应的寄存器，达到配置 ETH 工作环境的目的。

代码清单 43-1 ETH_InitTypeDef

```
1 typedef struct {  
2     /**
```

```

3      * @brief / * MAC
4      */
5      uint32_t  ETH_AutoNegotiation;           // 自适应功能
6      uint32_t  ETH_Watchdog;                  // 以太网看门狗
7      uint32_t  ETH_Jabber;                    // jabber 定时器功能
8      uint32_t  ETH_InterFrameGap;             // 发送帧间间隙
9      uint32_t  ETH_CarrierSense;              // 载波侦听
10     uint32_t  ETH_Speed;                      // 以太网速度
11     uint32_t  ETH_ReceiveOwn;                 // 接收自身
12     uint32_t  ETH_LoopbackMode;              // 回送模式
13     uint32_t  ETH_Mode;                      // 模式
14     uint32_t  ETH_ChecksumOffload;           // 校验和减荷
15     uint32_t  ETH_RetryTransmission;         // 传输重试
16     uint32_t  ETH_AutomaticPadCRCStrip;       // 自动去除 PAD 和 FCS 字段
17     uint32_t  ETH_BackOffLimit;              // 后退限制
18     uint32_t  ETH_DeferralCheck;             // 检查延迟
19     uint32_t  ETH_ReceiveAll;                // 接收所有 MAC 帧
20     uint32_t  ETH_SourceAddrFilter;          // 源地址过滤
21     uint32_t  ETH_PassControlFrames;         // 传送控制帧
22     uint32_t  ETH_BroadcastFramesReception;  // 广播帧接收
23     uint32_t  ETH_DestinationAddrFilter;     // 目标地址过滤
24     uint32_t  ETH_PromiscuousMode;           // 混合模式
25     uint32_t  ETH_MulticastFramesFilter;     // 多播源地址过滤
26     uint32_t  ETH_UnicastFramesFilter;       // 单播源地址过滤
27     uint32_t  ETH_HashTableHigh;            // 散列表高位
28     uint32_t  ETH_HashTableLow;             // 散列表低位
29     uint32_t  ETH_PauseTime;                 // 暂停时间
30     uint32_t  ETH_ZeroQuantaPause;           // 零时间片暂停
31     uint32_t  ETH_PauseLowThreshold;         // 暂停阈值下限
32     uint32_t  ETH_UnicastPauseFrameDetect;   // 单播暂停帧检测
33     uint32_t  ETH_ReceiveFlowControl;        // 接收流控制
34     uint32_t  ETH_TransmitFlowControl;       // 发送流控制
35     uint32_t  ETH_VLANTagComparison;         // VLAN 标记比较
36     uint32_t  ETH_VLANTagIdentifier;         // VLAN 标记标识符
37     /**
38     * @brief / * DMA
39     */
40     uint32_t  ETH_DropTCPIPChecksumErrorFrame; // 丢弃 TCP/IP 校验错误帧
41     uint32_t  ETH_ReceiveStoreForward;        // 接收存储并转发
42     uint32_t  ETH_FlushReceivedFrame;        // 刷新接收帧
43     uint32_t  ETH_TransmitStoreForward;       // 发送存储并转发
44     uint32_t  ETH_TransmitThresholdControl;   // 发送阈值控制
45     uint32_t  ETH_ForwardErrorFrames;        // 转发错误帧
46     uint32_t  ETH_ForwardUndersizedGoodFrames; // 转发过小的好帧
47     uint32_t  ETH_ReceiveThresholdControl;   // 接收阈值控制
48     uint32_t  ETH_SecondFrameOperate;        // 处理第二个帧
49     uint32_t  ETH_AddressAlignedBeats;       // 地址对齐节拍
50     uint32_t  ETH_FixedBurst;                // 固定突发
51     uint32_t  ETH_RxDMA BurstLength;         // DMA 突发接收长度
52     uint32_t  ETH_TxDMA BurstLength;         // DMA 突发发送长度
53     uint32_t  ETH_DescriptorSkipLength;     // 描述符跳过长度
54     uint32_t  ETH_DMA Arbitration;           // DMA 仲裁
55 } ETH_InitTypeDef;

```

- **ETH_AutoNegotiation:** 自适应功能选择，可选使能或禁止，一般选择使能自适应功能，系统会自动寻找最优工作方式，包括选择 10Mbps 或者 100Mbps 的以太网速度以及全双工模式或半双工模式。

- ❑ ETH_Watchdog: 以太网看门狗功能选择, 可选使能或禁止, 它设定以太网 MAC 配置寄存器(ETH_MACCR)的 WD 位的值。如果设置为 1, 使能看门狗, 在接收 MAC 帧超过 2048 字节时自动切断后面数据, 一般选择使能看门狗。如果设置为 0, 禁用看门狗, 最长可接收 16384 字节的帧。
- ❑ ETH_Jabber: jabber 定时器功能选择, 可选使能或禁止, 与看门狗功能类似, 只是看门狗用于接收 MAC 帧, jabber 定时器用于发送 MAC 帧, 它设定 ETH_MACCR 寄存器的 JD 位的值。如果设置为 1, 使能 jabber 定时器, 在发送 MAC 帧超过 2048 字节时自动切断后面数据, 一般选择使能 jabber 定时器。
- ❑ ETH_InterFrameGap: 控制发送帧间的最小间隙, 可选 96bit 时间、88bit 时间、...、40bit 时间, 他设定 ETH_MACCR 寄存器的 IFG[2:0]位的值, 一般设置 96bit 时间。
- ❑ ETH_CarrierSense: 载波侦听功能选择, 可选使能或禁止, 它设定 ETH_MACCR 寄存器的 CSD 位的值。当被设置为低电平时, MAC 发送器会生成载波侦听错误, 一般使能载波侦听功能。
- ❑ ETH_Speed: 以太网速度选择, 可选 10Mbps 或 100Mbit/s, 它设定 ETH_MACCR 寄存器的 FES 位的值, 一般设置 100Mbit/s, 但在使能自适应功能之后该位设置无效。
- ❑ ETH_ReceiveOwn: 接收自身帧功能选择, 可选使能或禁止, 它设定 ETH_MACCR 寄存器的 ROD 位的值, 当设置为 0 时, MAC 接收发送时 PHY 提供的所有 MAC 包, 如果设置为 1, MAC 禁止在半双工模式下接收帧。一般使能接收。
- ❑ ETH_LoopbackMode: 回送模式选择, 可选使能或禁止, 它设定 ETH_MACCR 寄存器的 LM 位的值, 当设置为 1 时, 使能 MAC 在 MII 回送模式下工作。
- ❑ ETH_Mode: 以太网工作模式选择, 可选全双工模式或半双工模式, 它设定 ETH_MACCR 寄存器 DM 位的值。一般选择全双工模式, 在使能了自适应功能后该成员设置无效。
- ❑ ETH_ChecksumOffload: IPv4 校验和减荷功能选择, 可选使能或禁止, 它设定 ETH_MACCR 寄存器 IPCO 位的值, 当该位被置 1 时使能接收的帧有效载荷的 TCP/UDP/ICMP 标头的 IPv4 校验和检查。一般选择禁用, 此时 PCE 和 IP HCE 状态位总是为 0。
- ❑ ETH_RetryTransmission: 传输重试功能, 可选使能或禁止, 它设定 ETH_MACCR 寄存器 RD 位的值, 当被设置为 1 时, MAC 仅尝试发送一次, 设置为 0 时, MAC 会尝试根据 BL 的设置进行重试。一般选择使能重试。
- ❑ ETH_AutomaticPadCRCStrip: 自动去除 PAD 和 FCS 字段功能, 可选使能或禁用, 它设定 ETH_MACCR 寄存器 APCS 位的值。当设置为 1 时, MAC 在长度字段值小于或等于 1500 自己是去除传入帧上的 PAD 和 FCS 字段。一般禁止自动去除 PAD 和 FCS 字段功能。
- ❑ ETH_BackOffLimit: 后退限制, 在发送冲突后重新安排发送的延迟时间, 可选 10、8、4、1, 它设定 ETH_MACCR 寄存器 BL 位的值。一般设置为 10。

- ❑ **ETH_DeferralCheck:** 检查延迟, 可选使能或禁止, 它设定 ETH_MACCCR 寄存器 DC 位的值, 当设置为 0 时, 禁止延迟检查功能, MAC 发送延迟, 直到 CRS 信号变成无效信号。
- ❑ **ETH_ReceiveAll:** 接收所有 MAC 帧, 可选使能或禁用, 它设定以太网 MAC 帧过滤寄存器(ETH_MACFFR)RA 位的值。当设置为 1 时, MAC 接收器将所有接收的帧传送到应用程序, 不过滤地址。当设置为 0 是, MAC 接收会自动过滤不与 SA/DA 匹配的帧。一般选择不接收所有。
- ❑ **ETH_SourceAddrFilter:** 源地址过滤, 可选源地址过滤、源地址反向过滤或禁用源地址过滤, 它设定 ETH_MACFFR 寄存器 SAF 位和 SAIF 位的值。一般选择禁用源地址过滤。
- ❑ **ETH_PassControlFrames:** 传送控制帧, 控制所有控制帧的转发, 可选阻止所有控制帧到达应用程序、转发所有控制帧、转发通过地址过滤的控制帧, 它设定 ETH_MACFFR 寄存器 PCF 位的值。一般选择禁止转发控制帧。
- ❑ **ETH_BroadcastFramesReception:** 广播帧接收, 可选使能或禁止, 它设定 ETH_MACFFR 寄存器 BFD 位的值。当设置为 0 时, 使能广播帧接收, 一般设置接收广播帧。
- ❑ **ETH_DestinationAddrFilter:** 目标地址过滤功能选择, 可选正常过滤或目标地址反向过滤, 它设定 ETH_MACFFR 寄存器 DAIF 位的值。一般设置为正常过滤。
- ❑ **ETH_PromiscuousMode:** 混合模式, 可选使能或禁用, 它设定 ETH_MACFFR 寄存器 PM 位的值。当设置为 1 时, 不论目标或源地址, 地址过滤器都传送所有传入的帧。一般禁用混合模式。
- ❑ **ETH_MulticastFramesFilter:** 多播源地址过滤, 可选完美散列表过滤、散列表过滤、完美过滤或禁用过滤, 它设定 ETH_MACFFR 寄存器 HPF 位、PAM 位和 HM 位的值。一般选择完美过滤。
- ❑ **ETH_UnicastFramesFilter:** 单播源地址过滤, 可选完美散列表过滤、散列表过滤或完美过滤, 它设定 ETH_MACFFR 寄存器 HPF 位和 HU 位的值。一般选择完美过滤。
- ❑ **ETH_HashTableHigh:** 散列表高位, 和 ETH_HashTableLow 组成 64 位散列表用于组地址过滤, 它设定以太网 MAC 散列表高位寄存器(ETH_MACHTHR)的值。
- ❑ **ETH_HashTableLow:** 散列表低位, 和 ETH_HashTableHigh 组成 64 位散列表用于组地址过滤, 它设定以太网 MAC 散列表低位寄存器(ETH_MACHTLR)的值。
- ❑ **ETH_PauseTime:** 暂停时间, 保留发送控制帧中暂停时间字段要使用的值, 可设置 0 至 65535, 它设定以太网 MAC 流控制寄存器(ETH_MACFCR)PT 位的值。
- ❑ **ETH_ZeroQuantaPause:** 零时间片暂停, 可选使用或禁止, 它设定 ETH_MACFCR 寄存器 ZQPD 位的值。当设置为 1 时, 当来自 FIFO 层的流控制信号去断言后, 此位会禁止自动生成零时间片暂停控制帧。一般选择禁止。
- ❑ **ETH_PauseLowThreshold:** 暂停阈值下限, 配置暂停定时器的阈值, 达到该值值时, 会自动程序传输暂停帧, 可选暂停时间减去 4 个间隙、28 个间隙、144 个间

隙或 256 个间隙，它设定 ETH_MACFCR 寄存器 PLT 位的值。一般选择暂停时间减去 4 个间隙。

- ❑ ETH_UnicastPauseFrameDetect：单播暂停帧检测，可选使能或禁止，它设定 ETH_MACFCR 寄存器 UPFD 位的值。当设置为 1 时，MAC 除了检测具有唯一多播地址的暂停帧外，还会检测具有 ETH_MACA0HR 和 ETH_MACA0LR 寄存器所指定的站单播地址的暂停帧。一般设置为禁止。
- ❑ ETH_ReceiveFlowControl：接收流控制，可选使能或禁止，它设定 ETH_MACFCR 寄存器 RFCE 位的值。当设置为 1 时，MAC 对接收到的暂停帧进行解码，并禁止其在指定时间（暂停时间）内发送；当设置为 0 时，将禁止暂停帧的解码功能，一般设置为禁止。
- ❑ ETH_TransmitFlowControl：发送流控制，可选使能或禁止，它设定 ETH_MACFCR 寄存器 TFCE 位的值。在全双工模式下，当设置为 1 时，MAC 将使能流控制操作来发送暂停帧；为 0 时，将禁止 MAC 中的流控制操作，MAC 不会传送任何暂停帧。在半双工模式下，当设置为 1 时，MAC 将使能背压操作；为 0 时，将禁止背压功能。
- ❑ ETH_VLANTagComparison：VLAN 标记比较，可选 12 位或 16 位，它设定以太网 MAC VLAN 标记寄存器(ETH_MACVLANTR)VLANTC 位的值。当设置为 1 时，使用 12 位 VLAN 标识符而不是完整的 16 位 VLAN 标记进行比较和过滤；为 0 时，使用全部 16 位进行比较，一般选择 16 位。
- ❑ ETH_VLANTagIdentifier：VLAN 标记标识符，包含用于标识 VLAN 帧的 802.1Q VLAN 标记，并与正在接收的 VLAN 帧的第十五和第十六字节进行比较。位 [15:13] 是用户优先级，位 [12] 是标准格式指示符(CFI)，位 [11:0] 是 VLAN 标记的 VLAN 标识符(VID)字段。VLANTC 位置 1 时，仅使用 VID（位 [11:0]）进行比较。
- ❑ ETH_DropTCPChecksumErrorFrame：丢弃 TCP/IP 校验错误帧，可选使能或禁止，它设定以太网 DMA 工作模式寄存器(ETH_DMAOMR)DTCEFD 位的值，当设置为 1 时，如果帧中仅存在由接收校验和减荷引擎检测出来的错误，则内核不会丢弃它；为 0 时，如果 FEF 为进行了复位，则会丢弃所有错误帧。
- ❑ ETH_ReceiveStoreForward：接收存储并转发，可选使能或禁止，它设定以太网 DMA 工作模式寄存器(ETH_DMAOMR)RSF 位的值，当设置为 1 时，向 RX FIFO 写入完整帧后可以从中读取一帧，同时忽略接收阈值控制(RTC)位；当设置为 0 时，RX FIFO 在直通模式下工作，取决于 RTC 位的阈值。一般选择使能。
- ❑ ETH_FlushReceivedFrame：刷新接收帧，可选使能或禁止，它设定 ETH_DMAOMR 寄存器 FTF 位的值，当设置为 1 时，发送 FIFO 控制器逻辑会恢复到缺省值，TX FIFO 中的所有数据均会丢失/刷新，刷新结束后改为自动清零。
- ❑ ETH_TransmitStoreForward：发送存储并转发，可选使能或禁止，它设定 ETH_DMAOMR 寄存器 TSF 位的值，当设置为 1 时，如果 TX FIFO 有一个完整的帧则发送会启动，会忽略 TTC 值；为 0 时，TTC 值才会有效。一般选择使能。

- ❑ **ETH_TransmitThresholdControl**：发送阈值控制，有多个阈值可选，它设定 ETH_DMAOMR 寄存器 TTC 位的值，当 TX FIFO 中帧大小大于该阈值时发送会自动，对于小于阈值的全帧也会发送。
- ❑ **ETH_ForwardErrorFrames**：转发错误帧，可选使能或禁止，它设定 ETH_DMAOMR 寄存器 FEF 位的值，当设置为 1 时，除了段错误帧之外所有帧都会转发到 DMA；为 0 时，RX FIFO 会丢弃滴啊有错误状态的帧。一般选择禁止。
- ❑ **ETH_ForwardUndersizedGoodFrames**：转发过小的好帧，可选使能或禁止，它设定 ETH_DMAOMR 寄存器 FUGF 位的值，当设置为 1 时，RX FIFO 会转发包括 PAD 和 FCS 字段的过小帧；为 0 时，会丢弃小于 64 字节的帧，除非接收阈值被设置为更低。
- ❑ **ETH_ReceiveThresholdControl**：接收阈值控制，当 RX FIFO 中的帧大小大于阈值时启动 DMA 传输请求，可选 64 字节、32 字节、96 字节或 128 字节，它设定 ETH_DMAOMR 寄存器 RTC 位的值。
- ❑ **ETH_SecondFrameOperate**：处理第二个帧，可选使能或禁止，它设定 ETH_DMAOMR 寄存器 OSF 位的值，当设置为 1 时会命令 DMA 处理第二个发送数据帧。
- ❑ **ETH_AddressAlignedBeats**：地址对齐节拍，可选使能或禁止，它设定以太网 DMA 总线模式寄存器(ETH_DMABMR)AAB 位的值，当设置为 1 并且固定突发位(FB)也为 1 时，AHB 接口会生成与起始地址 LS 位对齐的所有突发；如果 FB 位为 0，则第一个突发不对齐，但后续的突发与地址对齐。一般选择使能。
- ❑ **ETH_FixedBurst**：固定突发，控制 AHB 主接口是否执行固定突发传输，可选使能或禁止，它设定 ETH_DMABMR 寄存器 FB 位的值，当设置为 1 时，AHB 在正常突发传输开始期间使用 SINGLE、INCR4、INCR8 或 INCR16；为 0 时，AHB 使用 SINGLE 和 INCR 突发传输操作。
- ❑ **ETH_RxDMA BurstLength**：DMA 突发接收长度，有多个值可选，一般选择 32Beat，可实现 32*32bits 突发长度，它设定 ETH_DMABMR 寄存器 FPM 位和 RDP 位的值。
- ❑ **ETH_TxDMA BurstLength**：DMA 突发发送长度，有多个值可选，一般选择 32Beat，可实现 32*32bits 突发长度，它设定 ETH_DMABMR 寄存器 FPM 位和 PBL 位的值。
- ❑ **ETH_DescriptorSkipLength**：描述符跳过长度，指定两个未链接描述符之间跳过的字数，地址从当前描述符结束处开始跳到下一个描述符起始处，可选 0~7，它设定 ETH_DMABMR 寄存器 DSL 位的值。
- ❑ **ETH_DMA Arbitration**：DMA 仲裁，控制 RX 和 TX 优先级，可选 RX TX 优先级比为 1:1、2:1、3:1、4:1 或者 RX 优先于 TX，它设定 ETH_DMABMR 寄存器 PM 位和 DA 位的值，当设置为 1 时，RX 优先于 TX；为 0 时，循环调度，RX TX 优先级比由 PM 位给出。

43.8 以太网通信实验：无操作系统 LwIP 移植

LwIP 可以在带操作系统上运行，亦可在无操作系统上运行，这一实验我们讲解在无操作系统的移植步骤，并实现简单的传输代码，后续章节会讲解在带操作系统移植过程，一般都是在无操作系统基础上修改而来的。

43.8.1 硬件设计

在讲解移植步骤之前，有必须先介绍我们的实验硬件设计，主要是 LAN8720A 通过 RMII 和 SMI 接口与 STM32F4xx 控制器连接，见图 43-14。

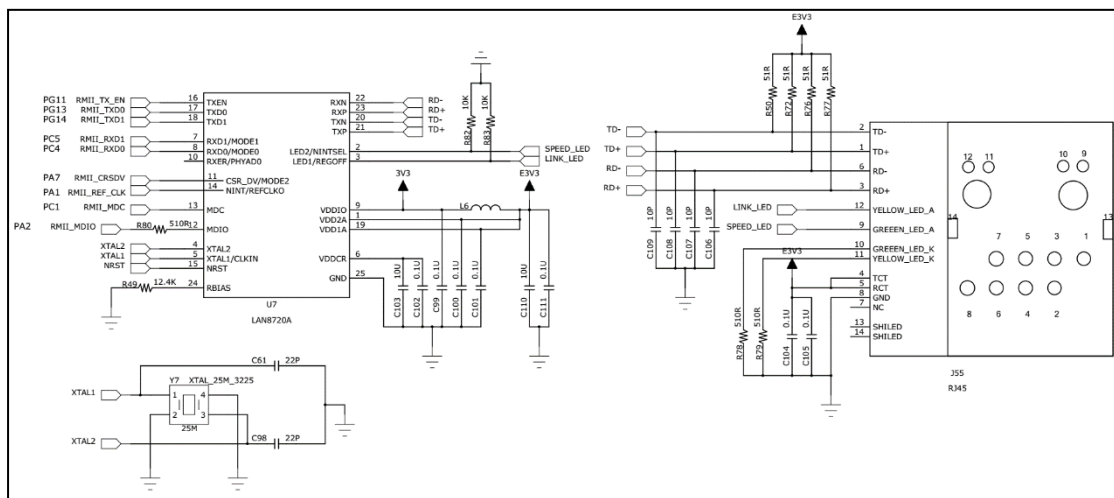


图 43-14 PHY 硬件设计

电路设计时，将 NINTSEL 引脚通过下拉电阻拉低，设置 NINT/FEFCLKO 为输出 50MHz 时钟，当然前提是在 XTAL1 和 XTAL2 接入了 25MHz 的时钟源。另外也把 REGOFF 引脚通过下拉电阻拉低，使能使用内部+1.2V 稳压器。

43.8.2 移植步骤

之前已经介绍了 LwIP 源代码(lwip-1.4.1.zip)和 ST 官方 LwIP 测试平台资料(stsw-stm32070.zip)下载，我们移植步骤是基于这两份资料进行的。

无操作系统移植 LwIP 需要的文件参考图 43-15，图中只显示了*.c 文件，还需要用到对应的*.h 文件。

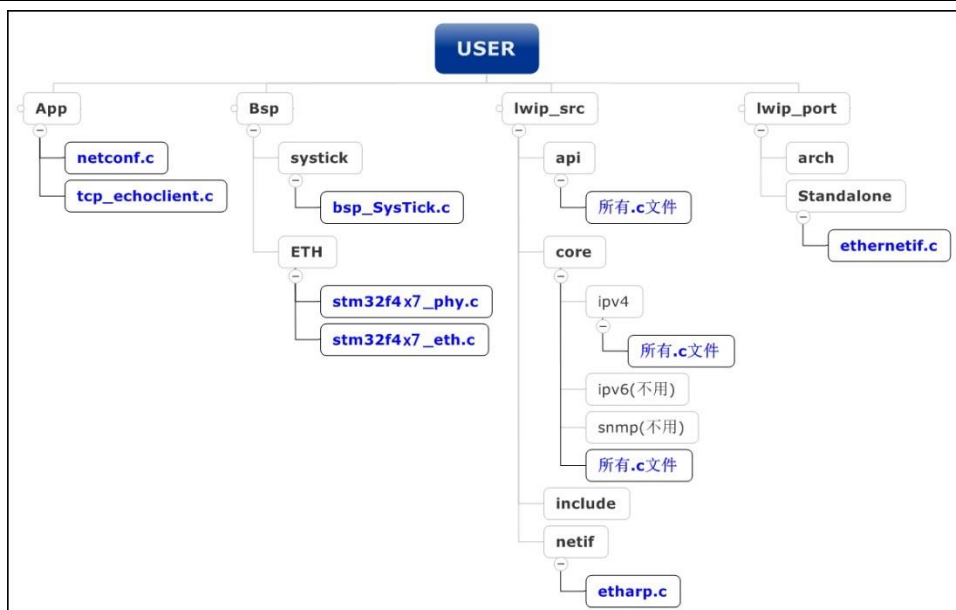


图 43-15 LwIP 移植实验文件结构

接下来，我们就根据图中文件结构详解移植过程。实验例程有需要用到系统滴答定时器 systick、调试串口 USART、独立按键 KEY、LED 灯功能，对这些功能实现不做具体介绍，可以参考相关章节理解。

第一步：相关文件拷贝

首先，解压 lwip-1.4.1.zip 和 stsw-stm32070.zip 两个压缩包，把整个 lwip-1.4.1 文件夹拷贝到 USER 文件夹下，特别说明，在整个移植过程中，不会对 lwip-1.4.1.zip 文件下的文件内容进行修改。然后，在 stsw-stm32070 文件夹找到 port 文件夹(路径：... \Utilities\Third_Party\lwip-1.4.1\port)，把整个 port 文件夹拷贝到 lwip-1.4.1 文件夹中，在 port 文件夹下的 STM32F4x7 文件夹中把 arch 和 Standalone 两个文件夹直接剪切到 port 文件夹中，即此时 port 文件夹有三个 STM32F4x7、arch 和 Standalone 文件夹，最后把 STM32F4x7 文件夹删除，最终的文件结构见图 43-16，arch 存放与开发平台相关头文件，Standalone 文件夹是无操作系统移植时 ETH 外设与 LwIP 连接的底层驱动函数。

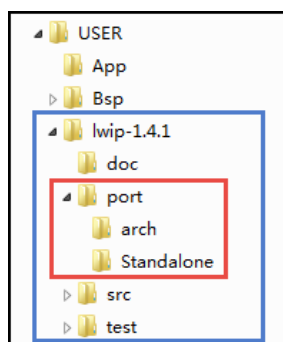


图 43-16 LwIP 相关文件拷贝

lwip-1.4.1 文件夹下的 doc 文件夹存放 LwIP 版权、移植、使用等等说明文件，移植之前有必须认真浏览一遍；src 文件夹存放 LwIP 的实现代码，也是我们工程代码真正需要的文件；test 文件夹存放 LwIP 部分功能测试例程；另外，还有一些无后缀名的文件，都是一

些说明性文件，可用记事本直接打开浏览。port 文件夹存放 LwIP 与 STM32 平台连接的相关文件，正如上面所说 contrib-1.4.1.zip 包含了不同平台移植代码，不过遗憾的是没有 STM32 平台的，所以我们需要从 ST 官方提供的测试平台找到这部分连接代码，也就是 port 文件夹的内容。

接下来，在 Bsp 文件下新建一个 ETH 文件夹，用于存放与 ETH 相关驱动文件，包括两个部分文件，其中一个 ETH 外设驱动文件，在 stsw-stm32070 文件夹中找到 stm32f4x7_eth.h 和 stm32f4x7_eth.c 两个文件(路径：...\Libraries\STM32F4x7_ETH_Driver)，将这两个文件拷贝到 ETH 文件夹中，这两个文件是 ETH 驱动文件，类似标准库中外设驱动代码实现文件，在移植过程中我们几乎不过文件的内容。这部分函数由 port 文件夹相关代码调用。另外一部分是相关 GPIO 初始化、ETH 外设初始化、PHY 状态获取等等函数的实现，在 stsw-stm32070 文件夹中找到 stm32f4x7_eth_bsp.c、stm32f4x7_eth_bsp.h 和 stm32f4x7_eth_conf.h 三个文件(路径：...\Project\Standalone\tcp_echo_client)，将这三个文件拷贝到 ETH 文件夹中。因为 ST 官方 LwIP 测试平台使用的 PHY 型号不是使用 LAN8720A，所以这三个文件需要我們进行修改。

最后，是 LwIP 测试代码实现，为测试 LwIP 移植是否成功和检查 LwIP 功能，我们编写 TCP 通信实现代码，设置开发板为 TCP 从机，电脑端为 TCP 主机。在 stsw-stm32070 文件夹中找到 netconf.c、tcp_echoclient.c、lwipopts.h、netconf.h 和 tcp_echoclient.h 五个文件(路径：...\Project\Standalone\tcp_echo_client)，直接拷贝到 App 文件夹(自己新建)中，netconf.c 文件代码实现 LwIP 初始化函数、周期调用函数、DHCP 功能函数等等，tcp_echoclient.c 文件实现 TCP 通信参数代码，lwipopts.h 包含 LwIP 功能选项。

第二部：为工程添加文件

第一步已经把相关的文件拷贝到对应的文件夹中，接下来就可以把需要用到的文件添加到工程中。图 43-15 已经指示出来工程需要用到的*.c 文件，所以最终工程文件结构见图 43-17，图中 api、ipv4 和 core 都包含了对应文件夹下的所有*.c 文件。

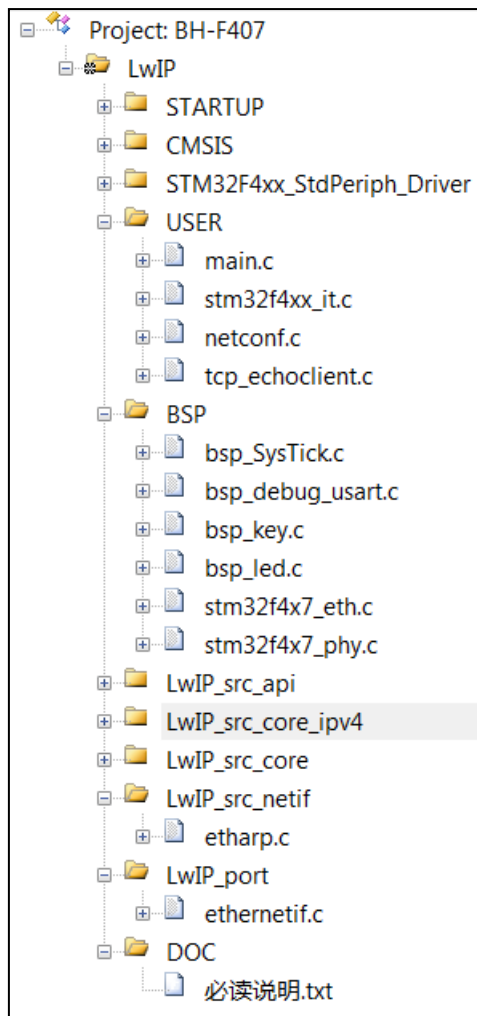


图 43-17 工程文件结构

接下来，还需要在工程选择中添加相关头文件路径，参考图 43-18。

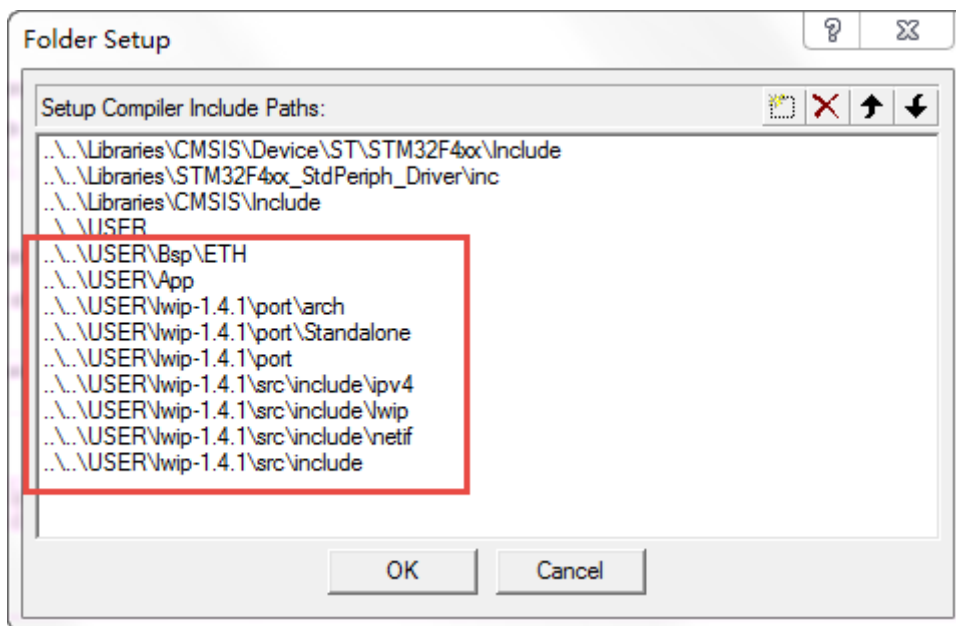


图 43-18 添加相关头文件路径

第三步：文件修改

ethernetif.c 文件是无操作系统时网络接口函数，该文件在移植是只需修改相关头文件名，函数实现部分无需修改。该文件主要有三个部分函数，一个是 low_level_init，用于初始化 MAC 相关工作环境、初始化 DMA 描述符链表，并使能 MAC 和 DMA；一个是 low_level_output，它是最底层发送一帧数据函数；最后一个是 low_level_input，它是最底层接收一帧数据函数。

stm32f4x7_eth.c 和 stm32f4x7_eth.h 两个文件用于 ETH 驱动函数实现，它是通过直接操作寄存器方式实现，这两个文件我们无需修改。stm32f4x7_eth_conf.h 文件包含了一些功能选项的宏定义，我们对部分内容进行了修改。

代码清单 43-2 stm32f4x7_eth_conf.h 文件宏定义

```
1 #ifndef USE_Delay
2 #include "Bsp/systick/bsp_SysTick.h"
3 #define _eth_delay_ Delay_10ms
4 #else
5 #define _eth_delay_ ETH_Delay
6 #endif
7
8 #ifndef USE_Delay
9 /* LAN8742A Reset delay */
10 #define LAN8742A_RESET_DELAY ((uint32_t)0x00000005)
11 #else
12 /* LAN8742A Reset delay */
13 #define LAN8742A_RESET_DELAY ((uint32_t)0x00FFFFFF)
14 #endif
15
16 /* The LAN8742A PHY status register */
17 /* PHY status register Offset */
18 #define PHY_SR ((uint16_t)0x001F)
19 /* PHY Speed mask 1:10Mb/s 0:100Mb/s*/
20 #define PHY_SPEED_STATUS ((uint16_t)0x0004)
21 /* PHY Duplex mask 1:Full duplex 0:Half duplex*/
22 #define PHY_DUPLEX_STATUS ((uint16_t)0x0010)
```

通过宏定义 USE_Delay 可选是否使用自定义的延时函数，Delay_10ms 函数是通过系统滴答定时器实现的延时函数，ETH_Delay 函数是 ETH 驱动自带的简单循环延时函数，延时函数实现方法不同，对形参要求不同。因为 ST 官方例程是基于 DP83848 型号的 PHY，而开发板的 PHY 型号是 LAN8720A。LAN8720A 复位时需要一段延时时间，这里需要定义延时时间长度，大约 50ms。驱动代码中需要获取 PHY 的速度和工作模式，LAN8720A 的 R31 是特殊控制/状态寄存器，包括指示以太网速度和工作模式的状态位。

stm32f4x7_phy.c 和 stm32f4x7_phy.h 两个文件是 ETH 外设相关的底层配置，包括 RMII 接口 GPIO 初始化、SMI 接口 GPIO 初始化、MAC 控制器工作环境配置，还有一些 PHY 的状态获取和控制修改函数。ST 官方例程文件包含了中断引脚的相关配置，主要用于指示接收到以太网帧，我们这里不需要使用，采用无限轮询方法检测接收状态。stm32f4x7_phy.h 文件存放相关宏定义，包含 RMII 和 SMI 引脚信息等宏定义，其中要特别说明的有一个宏，定义了 PHY 地址：ETHERNET_PHY_ADDRESS，这里根据硬件设计设置为 0x00，这在 SMI 通信是非常重要的。

代码清单 43-3 ETH_GPIO_Config 函数

```
1 void ETH_GPIO_Config(void)
2 {
3     GPIO_InitTypeDef GPIO_InitStructure;
```

```
4
5  /* Enable GPIOs clocks */
6  RCC_AHB1PeriphClockCmd(ETH_MDIO_GPIO_CLK | ETH_MDC_GPIO_CLK |
7      ETH_RMII_REF_CLK_GPIO_CLK|ETH_RMII_CRS_DV_GPIO_CLK|
8      ETH_RMII_RXD0_GPIO_CLK | ETH_RMII_RXD1_GPIO_CLK |
9      ETH_RMII_TX_EN_GPIO_CLK | ETH_RMII_TXD0_GPIO_CLK |
10     ETH_RMII_TXD1_GPIO_CLK , ENABLE);
11
12  /* Enable SYSCFG clock */
13  RCC_APB2PeriphClockCmd(RCC_APB2Periph_SYSCFG, ENABLE);
14
15  /* MII/RMII Media interface selection -----
16  */
17  #ifdef MII_MODE /* Mode MII with STM324xx-EVAL */
18  #ifdef PHY_CLOCK_MCO
19      /* Output HSE clock (25MHz) on MCO pin (PA8) to clock the PHY */
20      RCC_MCO1Config(RCC_MCO1Source_HSE, RCC_MCO1Div_1);
21  #endif /* PHY_CLOCK_MCO */
22  SYSCFG_ETH_MediaInterfaceConfig(SYSCFG_ETH_MediaInterface_MII);
23  #elif defined RMII_MODE /* Mode RMII with STM324xx-EVAL */
24
25  SYSCFG_ETH_MediaInterfaceConfig(SYSCFG_ETH_MediaInterface_RMII);
26  #endif
27
28  /* Ethernet pins configuration *****/
29  /*
30      ETH_MDIO -----> PA2
31      ETH_MDC -----> PC1
32      ETH_MII_RX_CLK/ETH_RMII_REF_CLK---> PA1
33      ETH_MII_RX_DV/ETH_RMII_CRS_DV ----> PA7
34      ETH_MII_RXD0/ETH_RMII_RXD0 -----> PC4
35      ETH_MII_RXD1/ETH_RMII_RXD1 -----> PC5
36      ETH_MII_TX_EN/ETH_RMII_TX_EN ----> PB11
37      ETH_MII_TXD0/ETH_RMII_TXD0 -----> PG13
38      ETH_MII_TXD1/ETH_RMII_TXD1 -----> PG14
39      ETH_NRST -----> PI1
40
41      */
42
43  /* Configure ETH_MDIO */
44  GPIO_InitStructure.GPIO_Pin = ETH_MDIO_PIN;
45  GPIO_InitStructure.GPIO_Speed = GPIO_Speed_100MHz;
46  GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF;
47  GPIO_InitStructure.GPIO_OType = GPIO_OType_PP;
48  GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_NOPULL;
49  GPIO_Init(ETH_MDIO_PORT, &GPIO_InitStructure);
50  GPIO_PinAFConfig(ETH_MDIO_PORT, ETH_MDIO_SOURCE, ETH_MDIO_AF);
51
52  /* Configure ETH_MDC */
53  GPIO_InitStructure.GPIO_Pin = ETH_MDC_PIN;
54  GPIO_Init(ETH_MDC_PORT, &GPIO_InitStructure);
55  GPIO_PinAFConfig(ETH_MDC_PORT, ETH_MDC_SOURCE, ETH_MDC_AF);
56
57  /******
58  /**      省略部分引脚初始化      */
59  /******
60
61  /* Configure ETH_RMII_TXD1 */
62  GPIO_InitStructure.GPIO_Pin = ETH_RMII_TXD1_PIN;
63  GPIO_Init(ETH_RMII_TXD1_PORT, &GPIO_InitStructure);
64  GPIO_PinAFConfig(ETH_RMII_TXD1_PORT, ETH_RMII_TXD1_SOURCE,
65      ETH_RMII_TXD1_AF);
66 }
```

STM32f4xx 控制器支持 MII 和 RMII 接口，通过程序控制使用 RMII 接口，同时需要使能 SYSCFG 时钟，函数后部分就是接口 GPIO 初始化实现，这里我们还连接了 LAN8720A 的复位引脚，通过拉低一段时间让芯片硬件复位。

代码清单 43-4 ETH_MACDMA_Config 函数

```
1 static void ETH_MACDMA_Config(void)
2 {
3     /* Enable ETHERNET clock */
4     RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_ETH_MAC |
5         RCC_AHB1Periph_ETH_MAC_Tx|RCC_AHB1Periph_ETH_MAC_Rx, ENABLE);
6
7     /* Reset ETHERNET on AHB Bus */
8     ETH_DeInit();
9     /* Software reset */
10    ETH_SoftwareReset();
11    /* Wait for software reset */
12    while (ETH_GetSoftwareResetStatus() == SET);
13
14    /* ETHERNET Configuration -----*/
15    /* 缺省配置 ETH_InitStructure */
16    ETH_StructInit(&ETH_InitStructure);
17
18    /* Fill ETH_InitStructure parameters */
19    /*----- MAC -----*/
20    /* 开启网络自适应功能，速度和工作模式无需配置 */
21    ETH_InitStructure.ETH_AutoNegotiation = ETH_AutoNegotiation_Enable;
22    // ETH_InitStructure.ETH_AutoNegotiation = ETH_AutoNegotiation_Disable;
23    // ETH_InitStructure.ETH_Speed = ETH_Speed_10M;
24    // ETH_InitStructure.ETH_Mode = ETH_Mode_FullDuplex;
25    /* 关闭反馈 */
26    ETH_InitStructure.ETH_LoopbackMode = ETH_LoopbackMode_Disable;
27    /* 关闭重传功能 */
28    ETH_InitStructure.ETH_RetryTransmission=ETH_RetryTransmission_Disable;
29    /* 关闭自动去除 PDA/CRC 功能 */
30    ETH_InitStructure.ETH_AutomaticPadCRCStrip =
31        ETH_AutomaticPadCRCStrip_Disable;
32    /* 关闭接收所有的帧 */
33    ETH_InitStructure.ETH_ReceiveAll = ETH_ReceiveAll_Disable;
34    /* 允许接收所有广播帧 */
35    ETH_InitStructure.ETH_BroadcastFramesReception =
36        ETH_BroadcastFramesReception_Enable;
37    /* 关闭混合模式的地址过滤 */
38    ETH_InitStructure.ETH_PromiscuousMode = ETH_PromiscuousMode_Disable;
39    /* 对于组播地址使用完美地址过滤 */
40    ETH_InitStructure.ETH_MulticastFramesFilter =
41        ETH_MulticastFramesFilter_Perfect;
42    /* 对单播地址使用完美地址过滤 */
43    ETH_InitStructure.ETH_UnicastFramesFilter =
44        ETH_UnicastFramesFilter_Perfect;
45    #ifdef CHECKSUM_BY_HARDWARE
46    /* 开启 ipv4 和 TCP/UDP/ICMP 的帧校验和卸载 */
47    ETH_InitStructure.ETH_ChecksumOffload = ETH_ChecksumOffload_Enable;
48    #endif
49
50    /*----- DMA -----*/
51    /*当我们使用帧校验和卸载功能的时候，一定要使能存储转发模式，存储
52    转发模式中要保证整个帧存储在 FIFO 中，这样 MAC 能插入/识别出帧校验
53    值，当真校验正确的时候 DMA 就可以处理帧，否则就丢掉该帧*/
54    /* 开启丢弃 TCP/IP 错误帧 */
55    ETH_InitStructure.ETH_DropTCPIPChecksumErrorFrame =
56        ETH_DropTCPIPChecksumErrorFrame_Enable;
57    /* 开启接收数据的存储转发模式 */
58    ETH_InitStructure.ETH_ReceiveStoreForward =
59        ETH_ReceiveStoreForward_Enable;
60    /* 开启发送数据的存储转发模式 */
61    ETH_InitStructure.ETH_TransmitStoreForward =
62        ETH_TransmitStoreForward_Enable;
63
```

```
64  /* 禁止转发错误帧 */
65  ETH_InitStructure.ETH_ForwardErrorFrames =
66      ETH_ForwardErrorFrames_Disable;
67  /* 不转发过小的好帧 */
68  ETH_InitStructure.ETH_ForwardUndersizedGoodFrames =
69      ETH_ForwardUndersizedGoodFrames_Disable;
70  /* 打开处理第二帧功能 */
71  ETH_InitStructure.ETH_SecondFrameOperate =
72      ETH_SecondFrameOperate_Enable;
73  /* 开启 DMA 传输的地址对齐功能 */
74  ETH_InitStructure.ETH_AddressAlignedBeats =
75      ETH_AddressAlignedBeats_Enable;
76  /* 开启固定突发功能 */
77  ETH_InitStructure.ETH_FixedBurst = ETH_FixedBurst_Enable;
78  /* DMA 发送的最大突发长度为 32 个节拍 */
79  ETH_InitStructure.ETH_RxDMABurstLength = ETH_RxDMABurstLength_32Beat;
80  /* DMA 接收的最大突发长度为 32 个节拍 */
81  ETH_InitStructure.ETH_TxDMABurstLength = ETH_TxDMABurstLength_32Beat;
82  ETH_InitStructure.ETH_DMAArbitration =
83      ETH_DMAArbitration_RoundRobin_RxTx_2_1;
84
85  /* 配置 ETH */
86  EthStatus = ETH_Init(&ETH_InitStructure, ETHERNET_PHY_ADDRESS);
87 }
```

首先是使能 ETH 时钟，复位 ETH 配置。ETH_StructInit 函数用于初始化 ETH_InitTypeDef 结构体变量，会给每个成员赋予缺省值。接下来就是根据需要配置 ETH_InitTypeDef 结构体变量，关于结构体各个成员意义已在“ETH 初始化结构体详解”作了分析。最后调用 ETH_Init 函数完成配置，ETH_Init 函数有两个形参，一个是 ETH_InitTypeDef 结构体变量指针，第二个是 PHY 地址，函数还有一个返回值，用于指示初始化配置是否成功。

代码清单 43-5 ETH_BSP_Config 函数

```
1 #define GET_PHY_LINK_STATUS()
2     (ETH_ReadPHYRegister(ETHERNET_PHY_ADDRESS, PHY_BSR) & 0x00000004)
3
4 void ETH_BSP_Config(void)
5 {
6     /* Configure the GPIO ports for ethernet pins */
7     ETH_GPIO_Config();
8
9     /* Configure the Ethernet MAC/DMA */
10    ETH_MACDMA_Config();
11
12    /* Get Ethernet link status */
13    if (GET_PHY_LINK_STATUS()) {
14        EthStatus |= ETH_LINK_FLAG;
15    }
16 }
```

GET_PHY_LINK_STATUS() 是定义获取 PHY 链路状态的宏，如果 PHY 连接正常那么整个宏定义为 1，如果不正常则为 0，它是通过 ETH_ReadPHYRegister 函数读取 PHY 的基本状态寄存器(PHY_BSR)并检测其 Link Status 位得到的。

ETH_BSP_Config 函数分别调用 ETH_GPIO_Config 和 ETH_MACDMA_Config 函数完成 ETH 初始化配置，最后调用 GET_PHY_LINK_STATUS() 来判断 PHY 状态，并保存在 EthStatus 变量中。ETH_BSP_Config 函数一般在 main 函数中优先 LwIP_Init 函数调用。

代码清单 43-6 ETH_CheckLinkStatus 函数

```
1 void ETH_CheckLinkStatus(uint16_t PHYAddress)
```

```
2 {
3     static uint8_t status = 0;
4     uint32_t t = GET_PHY_LINK_STATUS();
5
6     /* If we have link and previous check was not yet */
7     if (t && !status) {
8         /* Set link up */
9         netif_set_link_up(&gnetif);
10
11         status = 1;
12     }
13     /* If we don't have link and it was on previous check */
14     if (!t && status) {
15         EthLinkStatus = 1;
16         /* Set link down */
17         netif_set_link_down(&gnetif);
18
19         status = 0;
20     }
21 }
```

ETH_CheckLinkStatus 函数用于获取 PHY 状态，实际上也是通过宏定义 GET_PHY_LINK_STATUS() 获取到的，函数还根据 PHY 状态通知 LwIP 当前链路状态，gnetif 是一个 netif 结构体类型变量，LwIP 定义了 netif 结构体类型，用于指示某一网卡相关信息，LwIP 是支持多个网卡设备，使用时需要为每个网卡设备定义一个 netif 类型变量。无操作系统时 ETH_CheckLinkStatus 函数被无限循环调用。

代码清单 43-7 ETH_link_callback 函数

```
1 void ETH_link_callback(struct netif *netif)
2 {
3     __IO uint32_t timeout = 0;
4     uint32_t tmpreg;
5     uint16_t RegValue;
6     struct ip_addr ipaddr;
7     struct ip_addr netmask;
8     struct ip_addr gw;
9
10    if (netif_is_link_up(netif)) {
11        /* Restart the auto-negotiation */
12        if (ETH_InitStructure.ETH_AutoNegotiation !=
13            ETH_AutoNegotiation_Disable) {
14            /* Reset Timeout counter */
15            timeout = 0;
16            /* Enable auto-negotiation */
17            ETH_WritePHYRegister(ETHERNET_PHY_ADDRESS, PHY_BCR,
18                                PHY_AutoNegotiation);
19            /* Wait until the auto-negotiation will be completed */
20            do {
21                timeout++;
22            } while (!(ETH_ReadPHYRegister(ETHERNET_PHY_ADDRESS, PHY_BSR)
23                & PHY_AutoNego_Complete) && (timeout < (uint32_t) PHY_READ_TO));
24
25            /* Reset Timeout counter */
26            timeout = 0;
27            /* Read the result of the auto-negotiation */
28            RegValue = ETH_ReadPHYRegister(ETHERNET_PHY_ADDRESS, PHY_SR);
29
30            if ((RegValue & PHY_DUPLEX_STATUS) != (uint16_t) RESET) {
31                ETH_InitStructure.ETH_Mode = ETH_Mode_FullDuplex;
32            } else {
33                ETH_InitStructure.ETH_Mode = ETH_Mode_HalfDuplex;
34            }
35            if (RegValue & PHY_SPEED_STATUS) {
36                /* Set Ethernet speed to 10M following the auto-negotiation */
37                ETH_InitStructure.ETH_Speed = ETH_Speed_10M;
```

```
38         } else {
39             /* Set Ethernet speed to 100M following the auto-negotiation */
40             ETH_InitStructure.ETH_Speed = ETH_Speed_100M;
41         }
42
43         /*----- ETHERNET MACCR Re-Configuration -----
44         */
45         /* Get the ETHERNET MACCR value */
46         tmpreg = ETH->MACCR;
47
48         /* Set the FES bit according to ETH_Speed value */
49         /* Set the DM bit according to ETH_Mode value */
50         tmpreg |= (uint32_t)(ETH_InitStructure.ETH_Speed |
51                               ETH_InitStructure.ETH_Mode);
52
53         /* Write to ETHERNET MACCR */
54         ETH->MACCR = (uint32_t)tmpreg;
55
56         _eth_delay(ETH_REG_WRITE_DELAY);
57         tmpreg = ETH->MACCR;
58         ETH->MACCR = tmpreg;
59     }
60
61     /* Restart MAC interface */
62     ETH_Start();
63
64     #ifdef USE_DHCP
65     ipaddr.addr = 0;
66     netmask.addr = 0;
67     gw.addr = 0;
68     DHCP_state = DHCP_START;
69 #else
70     IP4_ADDR(&ipaddr, IP_ADDR0, IP_ADDR1, IP_ADDR2, IP_ADDR3);
71     IP4_ADDR(&netmask, NETMASK_ADDR0, NETMASK_ADDR1,
72             NETMASK_ADDR2, NETMASK_ADDR3);
73     IP4_ADDR(&gw, GW_ADDR0, GW_ADDR1, GW_ADDR2, GW_ADDR3);
74 #endif /* USE_DHCP */
75
76     netif_set_addr(&gnetif, &ipaddr, &netmask, &gw);
77
78     /* When the netif is fully configured this function must be called.*/
79     netif_set_up(&gnetif);
80
81     EthLinkStatus = 0;
82 } else {
83     ETH_Stop();
84 #ifdef USE_DHCP
85     DHCP_state = DHCP_LINK_DOWN;
86     dhcp_stop(netif);
87 #endif /* USE_DHCP */
88
89     /* When the netif link is down this function must be called.*/
90     netif_set_down(&gnetif);
91 }
```

ETH_link_callback 函数被 LwIP 调用，当链路状态发生改变时该函数就被调用，用于状态改变后处理相关事务。首先调用 netif_is_link_up 函数判断新状态是否是链路启动状态，如果是启动状态就进入 if 语句，接下来会判断 ETH 是否被设置为自适应模式，如果不是自适应模式需要使用 ETH_WritePHYRegister 函数使能 PHY 工作为自适应模式，然后 ETH_ReadPHYRegister 函数读取 PHY 相关寄存器，获取 PHY 当前支持的以太网速度和工作模式，并保存到 ETH_InitStructure 结构体变量中。ETH_Start 函数用于使能 ETH 外设，

之后就是配置 ETH 的 IP 地址、子网掩码、网关，如果是定义了 DHCP (动态主机配置协议) 功能则启动 DHCP。最后就是调用 `netif_set_up` 函数在 LwIP 层次配置启动 ETH 功能。

如果检测到是链路关闭状态，调用 `ETH_Stop` 函数关闭 ETH，如果定义了 DHCP 功能则需关闭 DHCP，最后调用 `netif_set_down` 函数在 LwIP 层次关闭 ETH 功能。

以上对文件修改部分更多涉及到 ETH 硬件底层驱动，一些是 PHY 芯片驱动函数、一些是 ETH 外设与 LwIP 连接函数。接下来要讲解的文件代码更多是与 LwIP 应用相关的。

`netconf.c` 和 `netconf.h` 文件用于存放 LwIP 配置相关代码。`netcon.h` 定义了相关宏。

代码清单 43-8 LwIP 配置相关宏定义

```
1 /* DHCP 状态 */
2 #define DHCP_START 1
3 #define DHCP_WAIT_ADDRESS 2
4 #define DHCP_ADDRESS_ASSIGNED 3
5 #define DHCP_TIMEOUT 4
6 #define DHCP_LINK_DOWN 5
7
8 // #define USE_DHCP /* enable DHCP, if disabled static address is
used */
9
10 /* 调试信息输出 */
11 #define SERIAL_DEBUG
12 /* 远端 IP 地址和端口 */
13 #define DEST_IP_ADDR0 192
14 #define DEST_IP_ADDR1 168
15 #define DEST_IP_ADDR2 1
16 #define DEST_IP_ADDR3 105
17 #define DEST_PORT 6000
18
19 /* MAC 地址: 网卡地址 */
20 #define MAC_ADDR0 2
21 #define MAC_ADDR1 0
22 #define MAC_ADDR2 0
23 #define MAC_ADDR3 0
24 #define MAC_ADDR4 0
25 #define MAC_ADDR5 0
26
27 /* 静态 IP 地址 */
28 #define IP_ADDR0 192
29 #define IP_ADDR1 168
30 #define IP_ADDR2 1
31 #define IP_ADDR3 122
32
33 /* 子网掩码 */
34 #define NETMASK_ADDR0 255
35 #define NETMASK_ADDR1 255
36 #define NETMASK_ADDR2 255
37 #define NETMASK_ADDR3 0
38
39 /* 网关 */
40 #define GW_ADDR0 192
41 #define GW_ADDR1 168
42 #define GW_ADDR2 1
43 #define GW_ADDR3 1
44
45 /* 检测 PHY 链路状态的实际间隔 (单位: ms) */
46 #ifndef LINK_TIMER_INTERVAL
47 #define LINK_TIMER_INTERVAL 1000
48 #endif
49
50 /* MII and RMII mode selection *****/
51 #define RMII_MODE
```

```
52 // #define MII_MODE
53
54 /* 在 MII 模式时, 使能 MCO 引脚输出 25MHz 脉冲 */
55 #ifndef MII_MODE
56 #define PHY_CLOCK_MCO
57 #endif
```

USE_DHCP 宏用于定义是否使用 DHCP 功能, 如果不定义该宏, 直接使用静态的 IP 地址, 如果定义该宏, 则使用 DHCP 功能, 获取动态的 IP 地址, 这里有个需要注意的地方, 电脑是没办法提供 DHCP 服务功能的, 路由器才有 DHCP 服务功能, 使用当开发板直连电脑时不能定义该宏。

SERIAL_DEBUG 宏是定义是否使能串口定义相关调试信息功能, 一般选择使能, 所以在 main 函数中需要添加串口初始化函数。

接下来, 定义了远端 IP 和端口、MAC 地址、静态 IP 地址、子网掩码、网关相关宏, 可以根据实际情况修改。

LAN8720A 仅支持 RMII 接口, 根据硬件设计这里定义使用 RMII_MODE。

代码清单 43-9 LwIP_Init 函数

```
1 void LwIP_Init(void)
2 {
3     struct ip_addr ipaddr;
4     struct ip_addr netmask;
5     struct ip_addr gw;
6
7     /* Initializes the dynamic memory heap defined by MEM_SIZE.*/
8     mem_init();
9     /* Initializes the memory pools defined by MEMP_NUM_x.*/
10    memp_init();
11
12    #ifndef USE_DHCP
13        ipaddr.addr = 0;
14        netmask.addr = 0;
15        gw.addr = 0;
16    #else
17        IP4_ADDR(&ipaddr, IP_ADDR0, IP_ADDR1, IP_ADDR2, IP_ADDR3);
18        IP4_ADDR(&netmask, NETMASK_ADDR0, NETMASK_ADDR1,
19                NETMASK_ADDR2, NETMASK_ADDR3);
20        IP4_ADDR(&gw, GW_ADDR0, GW_ADDR1, GW_ADDR2, GW_ADDR3);
21    #endif
22    /* 添加以太网设备 */
23    netif_add(&gnetif, &ipaddr, &netmask, &gw, NULL,
24             &ethernetif_init, &ethernet_input);
25
26    /* 设置以太网设备为默认网卡 */
27    netif_set_default(&gnetif);
28
29    if (EthStatus == (ETH_INIT_FLAG | ETH_LINK_FLAG)) {
30        gnetif.flags |= NETIF_FLAG_LINK_UP;
31        /* 配置完成网卡后启动网卡 */
32        netif_set_up(&gnetif);
33    #ifndef USE_DHCP
34        DHCP_state = DHCP_START;
35    #else
36    #ifndef SERIAL_DEBUG
37        printf("\n Static IP address  \n");
38        printf("IP: %d.%d.%d.%d\n", IP_ADDR0, IP_ADDR1, IP_ADDR2, IP_ADDR3);
39        printf("NETMASK: %d.%d.%d.%d\n", NETMASK_ADDR0, NETMASK_ADDR1,
40                NETMASK_ADDR2, NETMASK_ADDR3);
41        printf("Gateway: %d.%d.%d.%d\n", GW_ADDR0, GW_ADDR1, GW_ADDR2, GW_ADDR3);
42    #endif /* SERIAL_DEBUG */
43    #endif /* USE_DHCP */
```

```
44     } else {
45         /* 当网络链路关闭时关闭网卡设备 */
46         netif_set_down(&gnetif);
47 #ifndef USE_DHCP
48         DHCP_state = DHCP_LINK_DOWN;
49 #endif /* USE_DHCP */
50 #ifdef SERIAL_DEBUG
51         printf("\n Network Cable is \n");
52         printf("      not connected \n");
53 #endif /* SERIAL_DEBUG */
54     }
55     /* 设置链路回调函数, 用于获取链路状态 */
56     netif_set_link_callback(&gnetif, ETH_link_callback);
57 }
```

LwIP_Init 函数用于初始化 LwIP 协议栈, 一般在 main 函数中调用。首先是内存相关初始化, mem_init 函数是动态内存堆初始化, memp_init 函数是存储池初始化, LwIP 是实现内存的高效利用, 内部需要不同形式的内存管理模式。

接下来为 ipaddr、netmask 和 gw 结构体变量赋值, 设置本地 IP 地址、子网掩码和网关, 如果使用 DHCP 功能直接赋值为 0 即可。netif_add 是以太网设备添加函数, 即向 LwIP 协议栈申请添加一个网卡设备, 函数有 7 个形参, 第一个为 netif 结构体类型变量指针, 这里赋值为 gnetif 地址, 该网卡设备属性就存放在 gnetif 变量中; 第二个为 ip_addr 结构体类型变量指针, 用于设置网卡 IP 地址; 第三个 ip_addr 结构体类型变量指针, 用于设置子网掩码; 第四个为 ip_addr 结构体类型变量指针, 用于设置网关; 第五个为 void 变量, 用户自定义字段, 一般不用直接赋值 NULL; 第六个为 netif_init_fn 类型函数指针, 用于指向网卡设备初始化函数, 这里赋值为指向 ethernetif_init 函数, 该函数在 ethernetif.c 文件定义, 初始化 LwIP 与 ETH 外设连接函数; 最后一个参数为 netif_input_fn 类型函数指针, 用于指向以太网帧接收函数, 这里赋值为指向 ethernet_input 函数, 该函数定义在 etharp.c 文件中。

netif_set_default 函数用于设置指定网卡为默认的网络通信设备。

在无硬件连接错误时, 调用 ETH_BSP_Config(优先 LwIP_Init 函数被调用)时会把 EthStatus 变量对应的 ETH_LINK_FLAG 位使能, 所以在 LwIP_INIT 函数中会执行 if 判断语句代码, 置位网卡设备标志位以及运行 netif_set_up 函数启动网卡设备。否则执行 netif_set_down 函数停止网卡设备。

最后, 根据需要调用 netif_set_link_callback 函数实在当链路状态发生改变时需要调用的回调函数配置。

代码清单 43-10 LwIP_Pkt_Handle 函数

```
1 void LwIP_Pkt_Handle(void)
2 {
3     /* 从以太网存储器读取一个以太网帧并将其发送给 LwIP */
4     ethernetif_input(&gnetif);
5 }
```

LwIP_Pkt_Handle 函数用于从以太网存储器读取一个以太网帧并将其发送给 LwIP, 它在接收到以太网帧时被调用, 它是直接调用 ethernetif_input 函数实现的, 该函数定义在 ethernetif.c 文件中。

代码清单 43-11 LwIP_Periodic_Handle 函数

```
1 void LwIP_Periodic_Handle(__IO uint32_t localtime)
2 {
3 #if LWIP_TCP
4     /* TCP periodic process every 250 ms */
```

```
5     if (localtime - TCPTimer >= TCP_TMR_INTERVAL) {
6         TCPTimer =  localtime;
7         tcp_tmr();
8     }
9 #endif
10
11     /* ARP periodic process every 5s */
12     if ((localtime - ARPTimer) >= ARP_TMR_INTERVAL) {
13         ARPTimer =  localtime;
14         etharp_tmr();
15     }
16
17     /* Check link status periodically */
18     if ((localtime - LinkTimer) >= LINK_TIMER_INTERVAL) {
19         ETH_CheckLinkStatus(ETHERNET_PHY_ADDRESS);
20         LinkTimer=localtime;
21     }
22
23 #ifdef USE_DHCP
24     /* Fine DHCP periodic process every 500ms */
25     if (localtime - DHCPfineTimer >= DHCP_FINE_TIMER_MSECS) {
26         DHCPfineTimer =  localtime;
27         dhcp_fine_tmr();
28         if ((DHCP_state != DHCP_ADDRESS_ASSIGNED) &&
29             (DHCP_state != DHCP_TIMEOUT) &&
30             (DHCP_state != DHCP_LINK_DOWN)) {
31 #ifdef SERIAL_DEBUG
32             LED1_TOGGLE;
33             printf("\nFine DHCP periodic process every 500ms\n");
34 #endif /* SERIAL_DEBUG */
35
36             /* process DHCP state machine */
37             LwIP_DHCP_Process_Handle();
38         }
39     }
40
41     /* DHCP Coarse periodic process every 60s */
42     if (localtime - DHCPcoarseTimer >= DHCP_COARSE_TIMER_MSECS) {
43         DHCPcoarseTimer =  localtime;
44         dhcp_coarse_tmr();
45     }
46
47 #endif
48 }
```

LwIP_Periodic_Handle 函数是一个必须被无限循环调用的 LwIP 支持函数，一般在 main 函数的无限循环中调用，主要功能是为 LwIP 各个模块提供时间并查询链路状态，该函数有一个形参，用于指示当前时间，单位为 ms。

对于 TCP 功能，每 250ms 执行一次 tcp_tmr 函数；对于 ARP(地址解析协议)，每 5s 执行一次 etharp_tmr 函数；对于链路状态检测，每 1s 执行一次 ETH_CheckLinkStatus 函数；对于 DHCP 功能，每 500ms 执行一次 dhcp_fine_tmr 函数，如果 DHCP 处于 DHCP_START 或 DHCP_WAIT_ADDRESS 状态就执行 LwIP_DHCP_Process_Handle 函数，对于 DHCP 功能，还有每 60s 执行一次 dhcp_coarse_tmr 函数。

代码清单 43-12 LwIP_DHCP_Process_Handle 函数

```
1 void LwIP_DHCP_Process_Handle(void)
2 {
3     struct ip_addr ipaddr;
4     struct ip_addr netmask;
5     struct ip_addr gw;
6
7     switch (DHCP_state) {
8     case DHCP_START: {
```

```
9      DHCP_state = DHCP_WAIT_ADDRESS;
10      dhcp_start(&gnetif);
11      /* IP address should be set to 0
12         every time we want to assign a new DHCP address */
13      IPaddress = 0;
14 #ifdef SERIAL_DEBUG
15      printf("\n      Looking for      \n");
16      printf("      DHCP server      \n");
17      printf("      please wait... \n");
18 #endif /* SERIAL_DEBUG */
19 }
20 break;
21
22 case DHCP_WAIT_ADDRESS: {
23     /* Read the new IP address */
24     IPaddress = gnetif.ip_addr.addr;
25
26     if (IPaddress!=0) {
27         DHCP_state = DHCP_ADDRESS_ASSIGNED;
28         /* Stop DHCP */
29         dhcp_stop(&gnetif);
30 #ifdef SERIAL_DEBUG
31         printf("\n      IP address assigned \n");
32         printf("      by a DHCP server      \n");
33         printf("IP: %d.%d.%d.%d\n", (uint8_t) (IPaddress),
34             (uint8_t) (IPaddress >> 8), (uint8_t) (IPaddress >> 16),
35             (uint8_t) (IPaddress >> 24));
36         printf("NETMASK: %d.%d.%d.%d\n", NETMASK_ADDR0, NETMASK_ADDR1,
37             NETMASK_ADDR2, NETMASK_ADDR3);
38         printf("Gateway: %d.%d.%d.%d\n", GW_ADDR0, GW_ADDR1,
39             GW_ADDR2, GW_ADDR3);
40         LED1_ON;
41 #endif /* SERIAL_DEBUG */
42     } else {
43         /* DHCP timeout */
44         if (gnetif.dhcp->tries > MAX_DHCP_TRIES) {
45             DHCP_state = DHCP_TIMEOUT;
46             /* Stop DHCP */
47             dhcp_stop(&gnetif);
48             /* Static address used */
49             IP4_ADDR(&ipaddr, IP_ADDR0 , IP_ADDR1 , IP_ADDR2 , IP_ADDR3 );
50             IP4_ADDR(&netmask, NETMASK_ADDR0, NETMASK_ADDR1,
51                 NETMASK_ADDR2, NETMASK_ADDR3);
52             IP4_ADDR(&gw, GW_ADDR0, GW_ADDR1, GW_ADDR2, GW_ADDR3);
53             netif_set_addr(&gnetif, &ipaddr , &netmask, &gw);
54 #ifdef SERIAL_DEBUG
55             printf("\n      DHCP timeout      \n");
56             printf("      Static IP address      \n");
57             printf("IP: %d.%d.%d.%d\n", IP_ADDR0, IP_ADDR1,
58                 IP_ADDR2, IP_ADDR3);
59             printf("NETMASK: %d.%d.%d.%d\n", NETMASK_ADDR0, NETMASK_ADDR1,
60                 NETMASK_ADDR2, NETMASK_ADDR3);
61             printf("Gateway: %d.%d.%d.%d\n", GW_ADDR0, GW_ADDR1,
62                 GW_ADDR2, GW_ADDR3);
63             LED1_ON;
64 #endif /* SERIAL_DEBUG */
65         }
66     }
67 }
68 break;
69 default:
70     break;
71 }
72 }
```

LwIP_DHCP_Process_Handle 函数用于执行 DHCP 功能，当 DHCP 状态为 DHCP_START 时，执行 dhcp_start 函数启动 DHCP 功能，LwIP 会向 DHCP 服务器申请分

配 IP 请求，并进入等待分配状态。当 DHCP 状态为 DHCP_WAIT_ADDRESS 时，先判断 IP 地址是否为 0，如果不为 0 说明已经有 IP 地址，DHCP 功能已经完成可以停止它；如果 IP 地址总是为 0，就需要判断是否超过最大等待时间，并提示出错。

lwipopts.h 文件存放一些宏定义，用于剪切 LwIP 功能，比如有无操作系统、内存空间分配、存储池分配、TCP 功能、DHCP 功能、UDP 功能选择等等。这里使用与 ST 官方例程相同配置即可。

LwIP 为使用者提供了两种应用程序接口(API 函数)来实现 TCP/IP 协议栈，一种是低水平、基于回调函数的 API，称为 RAW API，另外一种是高水平、连续的 API，称为 sequential API，sequential API 又有两种函数结构，一种是 Netconn，一种是 Socket，它与在电脑端使用的 BSD 标准的 Socket API 结构和原理是非常相似的。

接下来内容我们使用 RAW API 实现一个简单的 TCP 通信测试，ST 官方有提供相关的例程，我们对其内容稍作调整。代码内容存放在 tcp_echoclient.c 文件中。TCP 在各个层次处理过程见图 43-19。

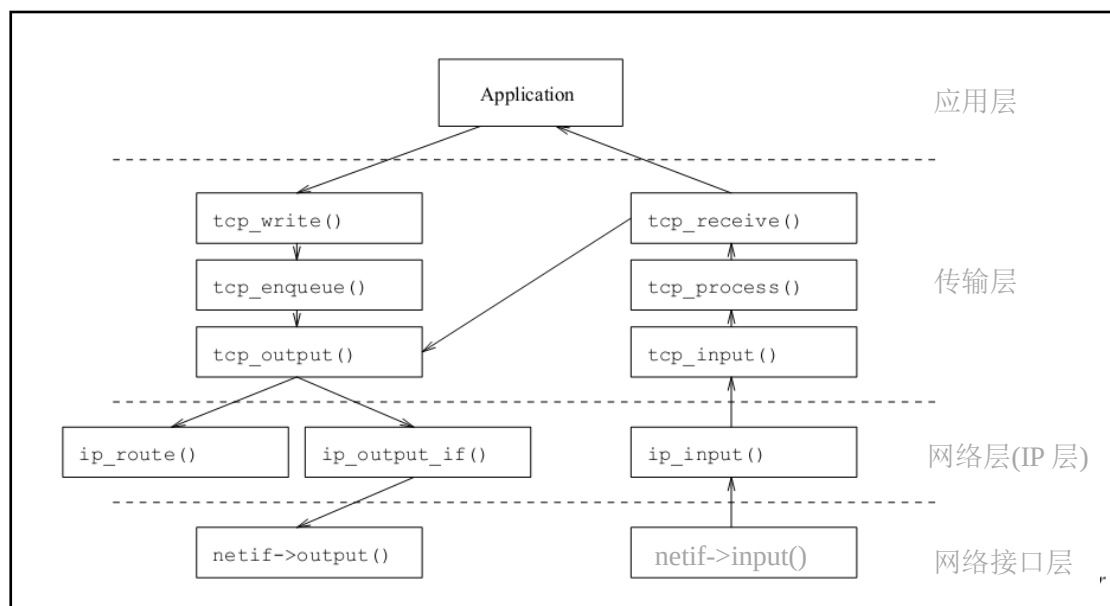


图 43-19 TCP 处理过程

网络接口层的 netif->output 和 netif->input 是在 ethernetif.c 文件中实现的，网络层和传输层有 LwIP 协议栈实现，应用层代码就是用户使用 LwIP 函数实现网络功能。

代码清单 43-13 tcp_echoclient_connect 函数

```

1 void tcp_echoclient_connect(void)
2 {
3     struct ip_addr DestIPAddr;
4
5     /* create new tcp pcb */
6     echoclient_pcb = tcp_new();
7
8     if (echoclient_pcb != NULL) {
9         IP4_ADDR(&DestIPAddr, DEST_IP_ADDR0, DEST_IP_ADDR1,
10                 DEST_IP_ADDR2, DEST_IP_ADDR3);
11
12         /* connect to destination address/port */
13         tcp_connect(echoclient_pcb, &DestIPAddr,
14                     DEST_PORT, tcp_echoclient_connected);
15     } else {

```



```
16      /* deallocate the pcb */
17      memp_free(MEMP_TCP_PCB, echoclient_pcb);
18 #ifdef SERIAL_DEBUG
19      printf("\n\r can not create tcp pcb");
20 #endif
21 }
22 }
```

tcp_echoclient_connect 函数用于创建 TCP 从设备并启动与 TCP 服务器连接。tcp_new 函数创建一个新 TCP 协议控制块，主要是必要的内存申请，返回一个未初始化的 TCP 协议控制块指针。如果返回值不为 0 就可以使用 tcp_connect 函数连接到 TCP 服务器，tcp_connect 函数用于 TCP 从设备连接至指定 IP 地址和端口的 TCP 服务器，它有四个形参，第一个为 TCP 协议控制块指针，第二个为服务器 IP 地址，第三个为服务器端口，第四个为函数指针，当连接正常建立时或连接错误时函数被调用，这里赋值 tcp_echoclient_connected 函数名。如果 tcp_new 返回值为 0 说明创建 TCP 协议控制块失败，调用 memp_free 函数释放相关内容。

代码清单 43-14 tcp_echoclient_disconnect 函数

```
1 struct echoclient {
2     enum echoclient_states state; /* connection status */
3     struct tcp_pcb *pcb;          /* pointer on the current tcp_pcb */
4     struct pbuf *p_tx;            /* pointer on pbuf to be transmitted */
5 };
6
7 void tcp_echoclient_disconnect(void)
8 {
9     /* close connection */
10    tcp_echoclient_connection_close(echoclient_pcb, echoclient_es);
11 #ifdef SERIAL_DEBUG
12    printf("\n\r close TCP connection");
13 #endif
14 }
```

echoclient 是自定义的一个结构体类型，包含了 TCP 从设备的状态、TCP 协议控制块指针和发送数据指针。tcp_echoclient_disconnect 函数用于断开 TCP 连接，通过调用 tcp_echoclient_connection_close 函数实现，它有两个形参，一个是 TCP 协议控制块，一个是 echoclient 类型指针。

代码清单 43-15 tcp_echoclient_connected 函数

```
1 static err_t tcp_echoclient_connected(void *arg, struct tcp_pcb *tpcb,
2                                           err_t err)
3 {
4     struct echoclient *es = NULL;
5
6     if (err == ERR_OK) {
7         /* allocate structure es to maintain tcp connection informations */
8         es = (struct echoclient *)mem_malloc(sizeof(struct echoclient));
9         echoclient_es=es;
10        if (es != NULL) {
11            es->state = ES_CONNECTED;
12            es->pcb = tpcb;
13            sprintf((char*)data, "sending tcp client message %d",
14                    message_count);
15            /* allocate pbuf */
16            es->p_tx = pbuf_alloc(PBUF_TRANSPORT, strlen((char*)data),
17                                  PBUF_POOL);
18            if (es->p_tx) {
19                /* copy data to pbuf */
20                pbuf_take(es->p_tx, (char*)data, strlen((char*)data));
21                /* pass newly allocated es structure as argument to tpcb */
```

```
22         tcp_arg(tpcb, es);
23         /* initialize LwIP tcp_recv callback function */
24         tcp_recv(tpcb, tcp_echoclient_recv);
25         /* initialize LwIP tcp_sent callback function */
26         tcp_sent(tpcb, tcp_echoclient_sent);
27         /* initialize LwIP tcp_poll callback function */
28         tcp_poll(tpcb, tcp_echoclient_poll, 1);
29         /* send data */
30         tcp_echoclient_send(tpcb, es);
31         return ERR_OK;
32     }
33     } else {
34         /* close connection */
35         tcp_echoclient_connection_close(tpcb, es);
36         /* return memory allocation error */
37         return ERR_MEM;
38     }
39     } else {
40         /* close connection */
41         tcp_echoclient_connection_close(tpcb, es);
42     }
43     return err;
44 }
```

tcp_echoclient_connected 函数作为 tcp_connect 函数设置的回调函数，在 TCP 建立连接时被调用，这里实现的功能是向 TCP 服务器发送一段数据。使用 mem_malloc 函数申请内存空间存放 echoclient 结构体类型数据，并赋值给 es 指针变量。如果内存申请失败调用 tcp_echoclient_connection_close 函数关闭 TCP 连接；确保内存申请成功后为 es 成员赋值，p_tx 成员是发送数据指针，这里使用 pbuf_alloc 函数向内存池申请存放发送数据的存储空间，即数据发送缓冲区。确保发送数据存储空间申请成功后使用 pbuf_take 函数将待发送数据 data 拷贝到数据发送存储器。tcp_arg 函数用于设置用户自定义参数，使得该参数可在相关回调函数被重新使用。tcp_recv、tcp_sent 和 tcp_poll 函数分别设置 TCP 协议控制块对应的接收、发送和轮询回调函数。最后调用 tcp_echoclient_send 函数发送数据。

代码清单 43-16 tcp_echoclient_recv 函数

```
1 static err_t tcp_echoclient_recv(void *arg, struct tcp_pcb *tpcb,
2                                     struct pbuf *p, err_t err)
3 {
4     char *recdata=0;
5     struct echoclient *es;
6     err_t ret_err;
7
8     LWIP_ASSERT("arg != NULL", arg != NULL);
9     es = (struct echoclient *)arg;
10    /* if we receive an empty tcp frame from server => close connection
11    */
12    if (p == NULL) {
13        /* remote host closed connection */
14        es->state = ES_CLOSING;
15        if (es->p_tx == NULL) {
16            /* we're done sending, close connection */
17            tcp_echoclient_connection_close(tpcb, es);
18        } else {
19            /* send remaining data*/
20            tcp_echoclient_send(tpcb, es);
21        }
22        ret_err = ERR_OK;
23    }
24    /* else : a non empty frame was received from echo server
25    but for some reason err != ERR_OK */
26    else if (err != ERR_OK) {
27        /* free received pbuf*/
```

```
27     pbuf_free(p);
28     ret_err = err;
29 } else if (es->state == ES_CONNECTED) {
30     /* increment message count */
31     message_count++;
32     /* Acknowledge data reception */
33     tcp_recved(tpcb, p->tot_len);
34 #ifdef SERIAL_DEBUG
35     recdata=(char *)malloc(p->len*sizeof(char));
36     if (recdata!=NULL) {
37         memcpy(recdata,p->payload,p->len);
38         printf("upd_rec<<%s",recdata);
39     }
40     free(recdata);
41 #endif
42     /* free received pbuf*/
43     pbuf_free(p);
44     ret_err = ERR_OK;
45 }
46 /* data received when connection already closed */
47 else {
48     /* Acknowledge data reception */
49     tcp_recved(tpcb, p->tot_len);
50
51     /* free pbuf and do nothing */
52     pbuf_free(p);
53     ret_err = ERR_OK;
54 }
55 return ret_err;
56 }
```

tcp_echoclient_rcv 函数是 TCP 接收回调函数，TCP 从设备接收到数据时该函数就被运行一次，我们可以提取数据帧内容。函数先检测是否为空帧，如果为空帧则关闭 TCP 连接，然后检测是否发生传输错误，如果发送错误执行 pbuf_free 函数释放内存。检查无错误就可以调用 tcp_recved 函数接收数据，这样就可以提取接收到信息。最后调用 pbuf_free 函数释放相关内存。

代码清单 43-17 tcp_echoclient_send 函数

```
1 static void tcp_echoclient_send(struct tcp_pcb *tpcb, struct echoclient * es)
2 {
3     struct pbuf *ptr;
4     err_t wr_err = ERR_OK;
5
6     while ((wr_err == ERR_OK) &&
7            (es->p_tx != NULL) &&
8            (es->p_tx->len <= tcp_sndbuf(tpcb))) {
9
10        /* get pointer on pbuf from es structure */
11        ptr = es->p_tx;
12
13        /* enqueue data for transmission */
14        wr_err = tcp_write(tpcb, ptr->payload, ptr->len, 1);
15
16        if (wr_err == ERR_OK) {
17            /* continue with next pbuf in chain (if any) */
18            es->p_tx = ptr->next;
19
20            if (es->p_tx != NULL) {
21                /* increment reference count for es->p */
22                pbuf_ref(es->p_tx);
23            }
24
25            /* free pbuf: will free pbufs up to es->p
26               (because es->p has a reference count > 0) */
27            pbuf_free(ptr);
28        }
29    }
```

```
28         } else if (wr_err == ERR_MEM) {
29             /* we are low on memory, try later, defer to poll */
30             es->p_tx = ptr;
31         } else {
32             /* other problem ?? */
33         }
34     }
35 }
```

tcp_echoclient_send 函数用于 TCP 数据发送，它有两个形参，一个是 TCP 协议控制块结构体指针，一个是 echoclient 结构体指针。在判断待发送数据存在并不超过最大可用发送队列数据数后，执行 tcp_write 函数将待发送数据写入发送队列，由协议内核决定发送时机。

代码清单 43-18 tcp_echoclient_poll 函数

```
1 static err_t tcp_echoclient_poll(void *arg, struct tcp_pcb *tpcb)
2 {
3     err_t ret_err;
4     struct echoclient *es;
5
6     es = (struct echoclient *)arg;
7     if (es != NULL) {
8         if (es->p_tx != NULL) {
9             /* there is a remaining pbuf (chain) , try to send data */
10            tcp_echoclient_send(tpcb, es);
11        } else {
12            /* no remaining pbuf (chain) */
13            if (es->state == ES_CLOSING) {
14                /* close tcp connection */
15                tcp_echoclient_connection_close(tpcb, es);
16            }
17        }
18        ret_err = ERR_OK;
19    } else {
20        /* nothing to be done */
21        tcp_abort(tpcb);
22        ret_err = ERR_ABRT;
23    }
24    return ret_err;
25 }
```

tcp_echoclient_poll 函数是由 tcp_poll 函数指定的回调函数，它每 500ms 执行一次，函数检测是否有待发送数据，如果有就执行 tcp_echoclient_send 函数发送数据。

代码清单 43-19 tcp_echoclient_sent 函数

```
1 static err_t tcp_echoclient_sent(void *arg, struct tcp_pcb *tpcb, u16_t len)
2 {
3     struct echoclient *es;
4
5     LWIP_UNUSED_ARG(len);
6
7     es = (struct echoclient *)arg;
8
9     if (es->p_tx != NULL) {
10        /* still got pbufs to send */
11        tcp_echoclient_send(tpcb, es);
12    }
13
14    return ERR_OK;
15 }
```

tcp_echoclient_sent 函数是有 tcp_sent 函数指定的回调函数，当接收到远端设备发送应答信号时被调用，它实际是通过调用 tcp_echoclient_send 函数发送数据实现的。

代码清单 43-20 tcp_echoclient_connection_close 函数

```
1 static void tcp_echoclient_connection_close(struct tcp_pcb *tpcb,  
2                                           struct echoclient * es )  
3 {  
4     /* remove callbacks */  
5     tcp_recv(tpcb, NULL);  
6     tcp_sent(tpcb, NULL);  
7     tcp_poll(tpcb, NULL, 0);  
8  
9     if (es != NULL) {  
10         mem_free(es);  
11     }  
12     /* close tcp connection */  
13     tcp_close(tpcb);  
14 }
```

tcp_echoclient_connection_close 函数用于关闭 TCP 连接，将相关的回调函数解除，释放 es 变量内存，最后调用 tcp_close 函数关闭 TCP 连接，释放 TCP 协议控制块内存。

代码清单 43-21 定时器初始化配置及中断服务函数

```
1 /* 初始化配置 TIM3，使能每 10ms 发生一次中断 */  
2 static void TIM3_Config(uint16_t period, uint16_t prescaler)  
3 {  
4     TIM_TimeBaseInitTypeDef TIM_TimeBaseInitStructure;  
5     NVIC_InitTypeDef NVIC_InitStructure;  
6  
7     RCC_APB1PeriphClockCmd(RCC_APB1Periph_TIM3, ENABLE);    ///使能 TIM3 时钟  
8  
9     TIM_TimeBaseInitStructure.TIM_Prescaler=prescaler;    //定时器分频  
10    TIM_TimeBaseInitStructure.TIM_CounterMode=TIM_CounterMode_Up;  
11    TIM_TimeBaseInitStructure.TIM_Period=period;    //自动重装载值  
12    TIM_TimeBaseInitStructure.TIM_ClockDivision=TIM_CKD_DIV1;  
13  
14    TIM_TimeBaseInit(TIM3, &TIM_TimeBaseInitStructure);  
15  
16    TIM_ITConfig(TIM3, TIM_IT_Update, ENABLE);    //允许定时器 3 更新中断  
17    TIM_Cmd(TIM3, ENABLE);    //使能定时器 3  
18  
19    NVIC_InitStructure.NVIC_IRQChannel=TIM3_IRQn;    //定时器 3 中断  
20    NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority=0x01;  
21    NVIC_InitStructure.NVIC_IRQChannelSubPriority=0x03;  
22    NVIC_InitStructure.NVIC_IRQChannelCmd=ENABLE;  
23    NVIC_Init(&NVIC_InitStructure);  
24 }  
25  
26 /* TIM3 中断服务函数 */  
27 void TIM3_IRQHandler(void)  
28 {  
29     if (TIM_GetITStatus(TIM3, TIM_IT_Update) == SET) {    //溢出中断  
30         LocalTime+=10;    //10ms 增量  
31     }  
32     TIM_ClearITPendingBit(TIM3, TIM_IT_Update);    //清除中断标志位  
33 }
```

LwIP_Periodic_Handle 函数执行 LwIP 需要周期性执行函数，该所以我们需要为该函数提高一个时间基准，这里使用 TIM3 产生这个基准，初始化配置 TIM3 每 10ms 中断一次，在其中断服务函数中递增 LocalTime 变量值。

代码清单 43-22 main 函数

```
1 int main(void)  
2 {  
3     uint8_t flag=0;  
4     /* 初始化 LED */  
5     LED_GPIO_Config();  
6 }
```

```
7      /* 初始化按键 */
8      Key_GPIO_Config();
9
10     /* 初始化调试串口, 一般为串口 1 */
11     Debug_USART_Config();
12
13     /* 初始化系统滴答定时器 */
14     SysTick_Init();
15
16     TIM3_Config(999,899);//10ms 定时器
17     printf("以太网通信实现例程\n");
18
19     /* Configure ethernet (GPIOs, clocks, MAC, DMA) */
20     ETH_BSP_Config();
21     printf("PHY 初始化结束\n");
22
23     /* Initilaize the LwIP stack */
24     LwIP_Init();
25
26     printf("    KEY1: 启动 TCP 连接\n");
27     printf("    KEY2: 断开 TCP 连接\n");
28
29     /* IP 地址和端口可在 netconf.h 文件修改, 或者使用 DHCP 服务自动获取 IP
30     (需要路由器支持) */
31     printf("本地 IP 和端口: %d.%d.%d.%d\n",IP_ADDR0,IP_ADDR1,
32           IP_ADDR2,IP_ADDR3);
33     printf("远端 IP 和端口: %d.%d.%d.%d\n",DEST_IP_ADDR0, DEST_IP_ADDR1,
34           DEST_IP_ADDR2, DEST_IP_ADDR3,DEST_PORT);
35
36     while (1) {
37         if ((Key_Scan(KEY1_GPIO_PORT,KEY1_PIN)==KEY_ON) && (flag==0)) {
38             LED2_ON;
39             if (EthLinkStatus == 0) {
40                 printf("connect to tcp server\n");
41                 /*connect to tcp server */
42                 tcp_echoclient_connect();
43                 flag=1;
44             }
45         }
46         if ((Key_Scan(KEY2_GPIO_PORT,KEY2_PIN)==KEY_ON) && flag) {
47             LED2_OFF;
48             tcp_echoclient_disconnect();
49             flag=0;
50         }
51         /* check if any packet received */
52         if (ETH_CheckFrameReceived()) {
53             /* process received ethernet packet */
54             LwIP_Pkt_Handle();
55         }
56         /* handle periodic timers for LwIP */
57         LwIP_Periodic_Handle(LocalTime);
58     }
59 }
```

首先是初始化 LED 指示灯、按键、调试串口、系统滴答定时器, TIM3_Config 函数配置 10ms 定时并启动定时器, ETH_BSP_Config 函数初始化 ETH 相关 GPIO、配置 MAC 和 DMA 并获取 PHY 状态, LwIP_Init 函数初始化 LwIP 协议栈。进入无限循环函数, 不断检测按键状态, 如果 KEY1 被按下则调用 tcp_echoclient_connect 函数启动 TCP 连接, 如果 KEY2 被按下则调用 tcp_echoclient_disconnect 关闭 TCP 连接。ETH_CheckFrameReceived 函数用于检测是否接收到数据帧, 如果接收到数据帧则调用 LwIP_Pkt_Handle 函数将数据帧从缓冲区传入 LwIP。LwIP_Periodic_Handle 函数执行必须被周期调用的函数。

下载验证

保证开发板相关硬件连接正确，用 USB 线连接开发板“USB TO UART”接口跟电脑，在电脑端打开串口调试助手并配置好相关参数；使用网线连接开发板网口跟路由器，这里要求电脑连接在同一个路由器上，之所以使用路由器是这样连接方便，电脑端无需更多操作步骤，并且路由器可以提供 DHCP 服务器功能，而电脑是不行的，最后在电脑端打开网络调试助手软件，并设置相关参数，见图 43-20，调试助手的设置与 netconf.h 文件中相关宏定义是对应的，不同电脑设置情况可能不同。把编译好的程序下载到开发板。

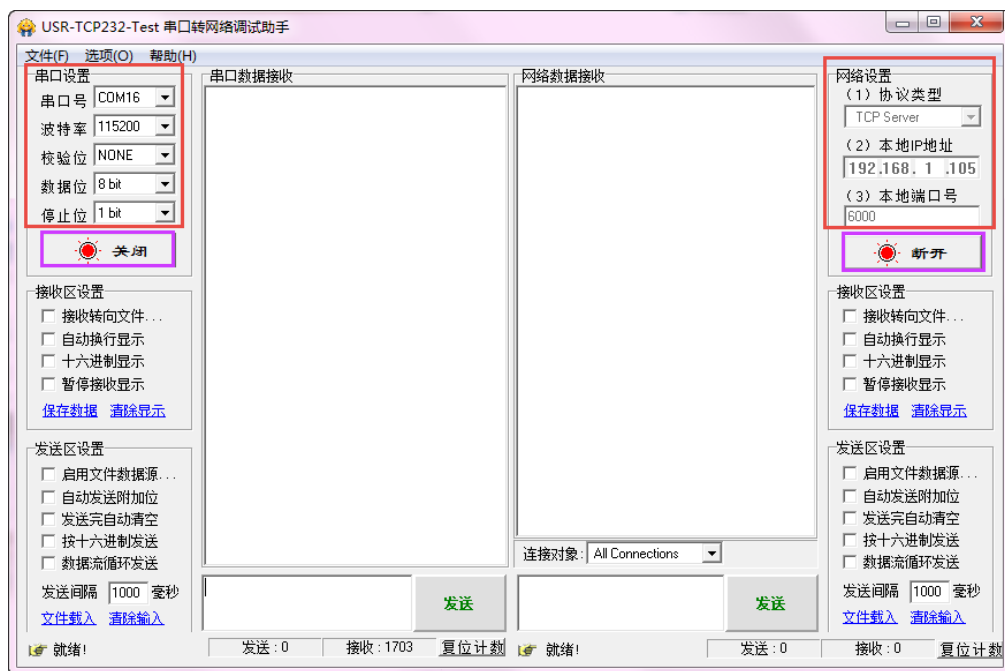


图 43-20 调试助手设置界面

在系统硬件初始化时串口调试助手会打印相关提示信息，等待初始化完成后可打开电脑端 CMD 窗口，输入 ping 命令测试开发板链路，图 43-21 为链路正常情况，如果出现 ping 不同情况，检查网线连接。

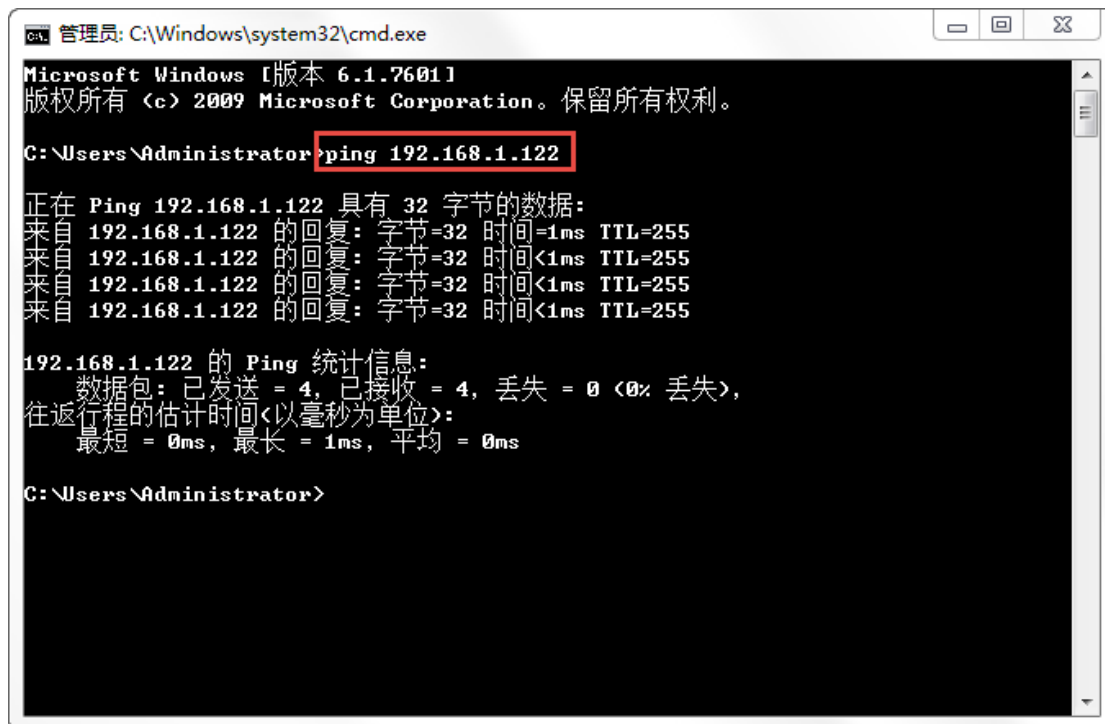


图 43-21 ping 窗口

ping 状态正常后，可按下开发板 KEY1 按键，使能开发板连接电脑端的 TCP 服务器，之后就可以进行数据传输，需要接收传输时可以按下开发板 KEY2 按键，实际操作调试助手界面见图 43-22。

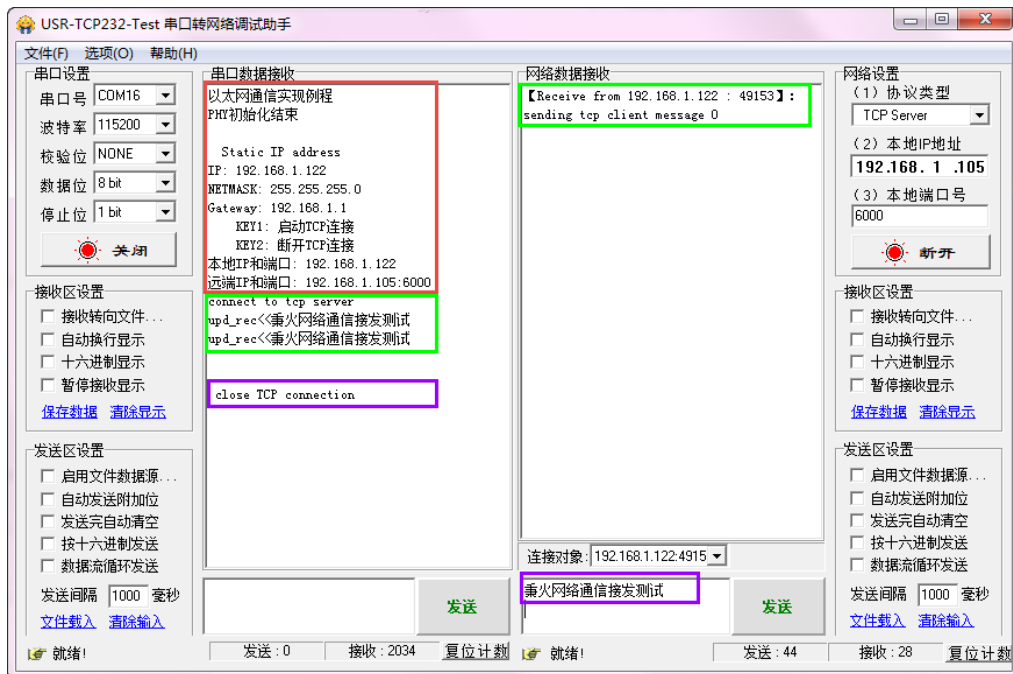


图 43-22 调试助手接发通信效果

43.9 基于 uCOS-III 移植 LwIP 实验

上面的实验是无操作系统移植 LwIP，LwIP 也确实支持无操作系统移植运行，这对于芯片资源紧张、不合适运行操作系统环境还是有很大用处的。不过在很多应用中会采用操作系统上运行 LwIP，这有利于提高整体性能。这个实验我们主要讲解移植操作步骤，过程中直接使用上个实验 LwIP 底层驱动，除非有需要修改地方才指出，同时这里假设已有移植好的 uCOS-III 工程可参考使用，关于 uCOS-III 移植部分可参考我们相关文档，这里主要介绍 LwIP 使用 uCOS-III 信号量、消息队列、定时器函数等等函数接口。

这个实验最终实现在 uCOS-III 操作系统基础上移植 LwIP，使能 DHCP 功能，在动态获取 IP 之后即可 ping 通。运行 uCOS-III 操作系统之后一般会使用 Netconn 或 Socket 方法使用 LwIP，关于这两个的应用方法限于篇幅问题这里不做深入探究。

UCOS-III 和 LwIP 都是属于软件编程层次，所以硬件设计部分并不需要做更改，直接使用上个实验的硬件设计即可。

接下来开始介绍移植步骤，为简化移植步骤，我们的思路是直接使用 uCOS-III 例程，在其基础上移植 LwIP 部分。

第一步：文件拷贝

拷贝整个 uCOS-III 工程，修改文件夹名称为“ETH—基于 uCOS-III 的 LwIP 移植”，作为我们这个实验工程基础，我们在此基础上添加功能。拷贝上个实验工程中的 lwip-1.4.1 整个文件夹到 USER 文件夹(路径：...\ETH—基于 uCOS-III 的 LwIP 移植\USER)中。

LwIP 源码部分，即 src 文件夹，内容是不用修改的，只有 port 文件夹内容需要修改。在 stsw-stm32070 文件夹找到 FreeRTOS 文件夹(路径：...\Utilities\Third_Party\lwip-1.4.1\port\STM32F4x7\FreeRTOS)，该文件夹内容是 LwIP 与 FreeRTOS 操作系统连接的相关接口函数，虽然我们选择使用 uCOS-III 操作系统，当还是有很多可以借鉴的地方，移植过程我们采用修改这些文件方法实现而不是完全自己新建文件，把 FreeRTOS 整个文件夹拷贝到 port 文件夹(路径：...\ETH—基于 uCOS-III 的 LwIP 移植\USER\lwip-1.4.1\port)内，并改名为 UCOS305，此时 port 文件夹内有三个文件夹，分别为：arch、Standalone、UCOS305，其中 Standalone 在本实验是不被使用的。

把上个实验工程中的 App 文件夹拷贝到本实验相同位置，其中 tcp_echoclient.c 和 tcp_echoclient.h 文件不是本实验需要的，将其删除。netconf.c、netconf.h 和 lwipopts.h 三个文件是必需的，但因为如果在本实验直接使用 lwipopts.h 文件需要修改较多地方，我们先将该文件删除，然后在 stsw-stm32070 文件夹找到 httpserver_socket 文件夹。(路径：...\Utilities\Third_Party\lwip-1.4.1\port\STM32F4x7\FreeRTOS\httpserver_socket)，在该文件夹下 inc 文件夹中的 lwipopts.h 文件是更方便我们移植的文件，我们拷贝它到 App 文件夹中。

最后，把上个实验工程中的 ETH 文件夹拷贝到本实验相同位置，这个文件夹内容都是必需的，但我们不用进行修改。

第二步：为工程添加文件

与上个工程相比，LwIP 部分文件只有 port 文件夹文件有所修改，其他使用与上个实验相同文件结构皆可，最终工程文件结构参考图 43-23。

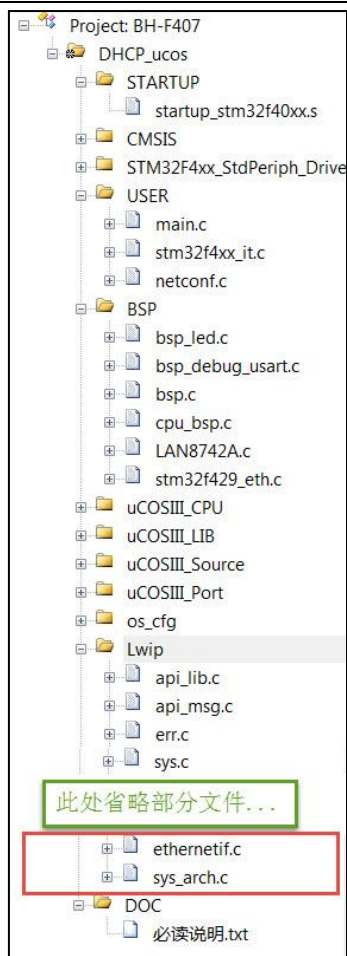


图 43-23 工程文件结构

添加完源文件后，还需要在工程选项中设置添加头文件路径，参考图 43-24。

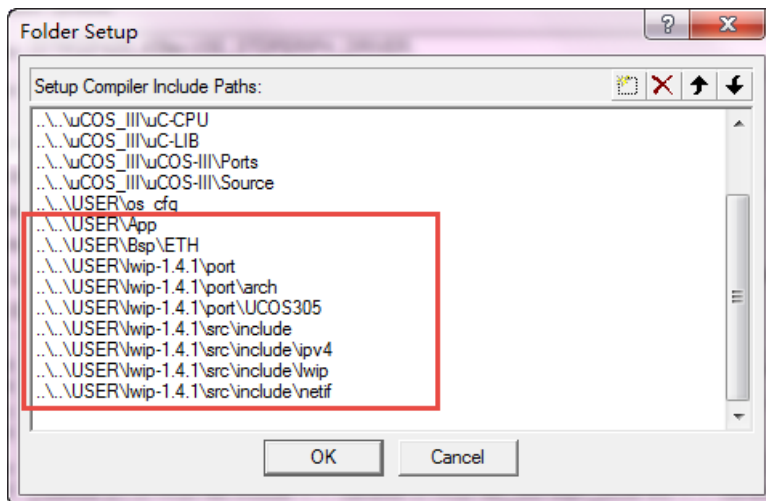


图 43-24 添加头文件路径

第三步：文件修改

ETH 文件夹内文件，stm32f4x7_eth.c、stm32f4x7_eth.h、stm32f4x7_phy.c 和 stm32f4x7_phy.h 四个文件是 ETH 外部和 PHY 相关驱动，本实验并无需修改硬件，所以这

四个文件内容不用修改，stm32f4x7_eth_conf.h 文件是与 ETH 外设相关硬件宏定义，因为本实验使用操作系统，对延时函数定义与上个实验工程有所不同，需要稍作修改。

代码清单 43-23 延时函数定义

```
1 #ifndef USE_Delay
2 #include "Bsp/bsp.h"
3 #define _eth_delay_ Delay_10ms
4 #else
5 #define _eth_delay_
6 #endif
```

这里使用在 bsp.h 文件中定义的 Delay_10ms 延时函数。

sys_arch.h 和 sys_arch.c 两个文件是 LwIP 与 uCOS-III 连接的实现代码。sys_arch.h 存放相关宏定义和类型定义。

代码清单 43-24 宏定义

```
1 #define LWIP_STK_SIZE 512
2 #define LWIP_TASK_MAX 8
3
4 #define LWIP_TSK_PRIO 3
5 #define LWIP_TASK_START_PRIO LWIP_TSK_PRIO
6 #define LWIP_TASK_END_PRIO LWIP_TSK_PRIO + LWIP_TASK_MAX
7
8 #define MAX_QUEUEUES 10 // 消息邮箱的数量
9 #define MAX_QUEUE_ENTRIES 20 // 每个邮箱的大小
10
11 #define SYS_MBOX_NULL (void *)0
12 #define SYS_SEM_NULL (void *)0
13
14 #define sys_arch_mbox_tryfetch(mbox,msg) sys_arch_mbox_fetch(mbox,msg,1)
```

宏 LWIP_STK_SIZE 定义 LwIP 任务栈空间大小，实际空间是 4*LWIP_STK_SIZE 个字节。宏 LWIP_TASK_MAX 定义预留给 LwIP 使用的最大任务数量。LWIP_TSK_PRIO、LWIP_TASK_START_PRIO 和 LWIP_TASK_END_PRIO 三个宏指定 LwIP 任务的优先级范围。宏 MAX_QUEUEUES 定义 LwIP 可以使用的最大邮箱数量，宏 MAX_QUEUE_ENTRIES 定义每个邮箱的大小。宏 SYS_MBOX_NULL 和 SYS_SEM_NULL 分别定义邮箱和信号量 NULL 对于的值。sys_arch_mbox_tryfetch 函数是尝试获取邮箱内容，这里直接调用 sys_arch_mbox_fetch 函数实现。

代码清单 43-25 类型定义

```
1 typedef OS_SEM sys_sem_t; // type of semiphores
2 typedef OS_MUTEX sys_mutex_t; // type of mutex
3 typedef OS_Q sys_mbox_t; // type of mailboxes
4 typedef CPU_INT08U sys_thread_t; // type of id of the new thread
5
6 typedef CPU_INT08U sys_prot_t;
```

不同操作系统有不同名称定义信号量、复合信号、邮箱、任务 ID 等等，这里使用 uCOS-III 操作系统需要使用对应的名称。

实际上，除了需要定于与操作系统对应的名称之外，还需要定于与编译器相关的名称，在 sys_arch.h 文件中有引用了 cc.h 头文件，因为我们使用 Windows 操作系统的 Keil 开发工具，需要对 cc.h 文件进行必须修改。

代码清单 43-26 编译器相关类型定于和宏定义

```
1 typedef u32_t mem_ptr_t;
2 //typedef int sys_prot_t;
3
```

```
4
5 // #define U16_F "hu"
6 // #define S16_F "d"
7 // #define X16_F "hx"
8 // #define U32_F "u"
9 // #define S32_F "d"
10 // #define X32_F "x"
11 // #define SZT_F "uz"
12
13 #define U16_F "4d"
14 #define S16_F "4d"
15 #define X16_F "4x"
16 #define U32_F "8ld"
17 #define S32_F "8ld"
18 #define X32_F "8lx"
```

sys_prot_t 类型已在 sys_arch.h 文件中定于, 在 cc.h 文件必须注释掉不被使用。U16_F、S16_F、X16_F 等等一系列名称用于 LwIP 的调试函数, 这一系列宏定于用于调试信息输出格式化。

代码清单 43-27 调试信息输出定于

```
1 #define LWIP_PLATFORM_DIAG(x) {printf x;}
2
3 #define LWIP_PLATFORM_ASSERT(x) do { printf("Assertion \"%s\" failed at \
4     line %d in %s\n", x, __LINE__, __FILE__); } while(0)
5
6 #define LWIP_ERROR(message, expression, handler) do { if (!(expression)) { \
7     printf("Assertion \"%s\" failed at line %d in %s\n", message, \
8     __LINE__, __FILE__); fflush(NULL); handler; } } while(0)
```

LwIP 实现代码已经添加了调试信息功能, 我们只需要定于信息输出途径即可, 这里直接使用 printf 函数, 将调试信息打印到串口调试助手。

sys_arch.c 文件存放 uCOS-III 与 LwIP 连接函数, LwIP 为实现在操作系统上运行, 预留了相关接口函数, 不同操作系统使用不同方法实现要求的功能。该文件存放在 UCOS305 文件夹内。

代码清单 43-28 sys_now 函数

```
1 u32_t sys_now()
2 {
3     OS_TICK os_tick_ctr;
4     CPU_SR_ALLOC();
5
6     CPU_CRITICAL_ENTER();
7     os_tick_ctr = OSTickCtr;
8     CPU_CRITICAL_EXIT();
9
10    return os_tick_ctr;
11 }
```

sys_now 函数用于为 LwIP 提供系统时钟, 这里直接的读取 OSTickCtr 变量值。

CPU_CRITICAL_ENTER 和 CPU_CRITICAL_EXIT 分别是关闭总中断和开启总中断。

LwIP 的邮箱用于缓存和传递数据包。

代码清单 43-29 邮箱创建与删除

```
1 err_t sys_mbox_new(sys_mbox_t *mbox, int size)
2 {
3     OS_ERR ucErr;
4
5     OSQCreate(mbox, "LWIP queue", size, &ucErr);
6     LWIP_ASSERT( "OSQCreate ", ucErr == OS_ERR_NONE );
```



```
7
8     if ( ucErr == OS_ERR_NONE) {
9         return 0;
10    }
11    return -1;
12 }
13
14 void sys_mbox_free(sys_mbox_t *mbox)
15 {
16     OS_ERR      ucErr;
17     LWIP_ASSERT( "sys_mbox_free ", mbox != SYS_MBOX_NULL );
18
19     OSQFlush(mbox,& ucErr);
20
21     OSQDel(mbox, OS_OPT_DEL_ALWAYS, &ucErr);
22     LWIP_ASSERT( "OSQDel ", ucErr == OS_ERR_NONE );
23 }
```

sys_mbox_new 函数要求实现的功能是创建一个邮箱，这里使用 OSQCreate 函数创建一个队列。sys_mbox_free 函数要求实现的功能是释放一个邮箱，如果邮箱存在内容，会发生错误，这里先使用 OSQFlush 函数清除队列内容，然后再使用 OSQDel 函数删除队列。

LWIP_ASSERT 函数是由 LwIP 定义的断言，用于调试错误。

代码清单 43-30 邮箱发送和获取

```
1 void sys_mbox_post(sys_mbox_t *mbox, void *data)
2 {
3     OS_ERR      ucErr;
4     CPU_INT08U  i=0;
5     if ( data == NULL ) data = (void*)&pvNullPointer;
6     /* try 10 times */
7     while (i<10) {
8         OSQPost(mbox, data,0,OS_OPT_POST_ALL,&ucErr);
9         if (ucErr == OS_ERR_NONE)
10             break;
11         i++;
12         OSTimeDly(5,OS_OPT_TIME_DLY,&ucErr);
13     }
14     LWIP_ASSERT( "sys_mbox_post error!\n", i !=10 );
15 }
16
17 err_t sys_mbox_trypost(sys_mbox_t *mbox, void *msg)
18 {
19     OS_ERR      ucErr;
20     if (msg == NULL ) msg = (void*)&pvNullPointer;
21     OSQPost(mbox, msg,0,OS_OPT_POST_ALL,&ucErr);
22     if (ucErr != OS_ERR_NONE) {
23         return ERR_MEM;
24     }
25     return ERR_OK;
26 }
27
28 u32_t sys_arch_mbox_fetch(sys_mbox_t *mbox, void **msg, u32_t timeout)
29 {
30     OS_ERR      ucErr;
31     OS_MSG_SIZE  msg_size;
32     CPU_TS      uc_timeout;
33     CPU_TS      in_timeout = timeout/LWIP_ARCH_TICK_PER_MS;
34     if (timeout && in_timeout == 0)
35         in_timeout = 1;
36     *msg = OSQPend (mbox,in_timeout,OS_OPT_PEND_BLOCKING,&msg_size,
37                    &uc_timeout,&ucErr);
38
39     if ( ucErr == OS_ERR_TIMEOUT )
40         uc_timeout = SYS_ARCH_TIMEOUT;
41     return uc_timeout;
42 }
```

sys_mbox_post 函数要求实现的功能是发送一个邮箱，这里主要调用 OSQPost 函数实现队列发送，为保证发送成功，最多尝试 10 次队列发送。sys_mbox_trypost 函数是尝试发送一个邮箱，这里我们直接使用 OSQPost 函数发送一次信号量，而不像 sys_mbox_post 函数在发送失败时可能尝试发送多次。sys_arch_mbox_fetch 函数用于获取邮箱内容，并指定等待超时时间，这里主要通过调用 OSQPend 函数实现队列获取。

代码清单 43-31 邮箱可用性检查和不可用设置

```
1 int sys_mbox_valid(sys_mbox_t *mbox)
2 {
3     if (mbox->NamePtr)
4         return (strcmp(mbox->NamePtr, "?Q"))? 1:0;
5     else
6         return 0;
7 }
8
9 void sys_mbox_set_invalid(sys_mbox_t *mbox)
10 {
11     if (sys_mbox_valid(mbox))
12         sys_mbox_free(mbox);
13 }
```

sys_mbox_valid 函数要求实现的功能是检查指定的邮箱是否可用，对于 uCOSIII，直接调用 strcmp 函数检查队列名称是否存在“Q”字段，如果存在说明该邮箱可用，否则不可用。sys_mbox_set_invalid 函数要求实现的功能是将指定的邮箱设置为不可用(无效)，这里先调用 sys_mbox_valid 函数判断邮箱是可用的，如果本身不可用就无需操作，确定邮箱可用后调用 sys_mbox_free 函数删除邮箱。

LwIP 的信号量用于进程间的通信。

代码清单 43-32 新建信号量

```
1 err_t sys_sem_new(sys_sem_t *sem, u8_t count)
2 {
3     OS_ERR ucErr;
4     OSSemCreate (sem, "LWIP Sem", count, &ucErr);
5     if (ucErr != OS_ERR_NONE ) {
6         LWIP_ASSERT("OSSemCreate ", ucErr == OS_ERR_NONE );
7         return -1;
8     }
9     return 0;
10 }
```

sys_sem_new 函数要求实现的功能是新建一个信号量，这里直接调用 OSSemCreate 函数新建一个信号量，count 参数用于指定信号量初始值。

代码清单 43-33 信号量相关函数

```
1 u32_t sys_arch_sem_wait(sys_sem_t *sem, u32_t timeout)
2 {
3     OS_ERR ucErr;
4     CPU_TS ucos_timeout;
5     CPU_TS in_timeout = timeout/LWIP_ARCH_TICK_PER_MS;
6     if (timeout && in_timeout == 0)
7         in_timeout = 1;
8     OSSemPend (sem, in_timeout, OS_OPT_PEND_BLOCKING, &ucos_timeout, &ucErr);
9     /* only when timeout! */
10    if (ucErr == OS_ERR_TIMEOUT)
11        ucos_timeout = SYS_ARCH_TIMEOUT;
12    return ucos_timeout;
13 }
14
15 void sys_sem_signal(sys_sem_t *sem)
16 {
```

```
17     OS_ERR ucErr;  
18     OSSemPost(sem, OS_OPT_POST_ALL, &ucErr);  
19     LWIP_ASSERT("OSSemPost ", ucErr == OS_ERR_NONE );  
20 }  
21  
22 void sys_sem_free(sys_sem_t *sem)  
23 {  
24     OS_ERR ucErr;  
25     OSSemDel(sem, OS_OPT_DEL_ALWAYS, &ucErr );  
26     LWIP_ASSERT("OSSemDel ", ucErr == OS_ERR_NONE );  
27 }  
28  
29 int sys_sem_valid(sys_sem_t *sem)  
30 {  
31     if (sem->NamePtr)  
32         return (strcmp(sem->NamePtr, "?SEM"))? 1:0;  
33     else  
34         return 0;  
35 }  
36  
37 void sys_sem_set_invalid(sys_sem_t *sem)  
38 {  
39     if (sys_sem_valid(sem))  
40         sys_sem_free(sem);  
41 }
```

sys_arch_sem_wait 函数要求实现的功能是等待获取一个信号量，并具有超时等待检查功能，这里直接调用 OSSemPend 函数实现信号量获取。sys_sem_signal 函数要求实现的功能是发送一个信号量，这里直接调用 OSSemPost 函数发送一个信号量。sys_sem_free 函数要求实现的功能是释放一个信号量，这里直接调用 OSSemDel 函数删除信号量。

sys_sem_valid 函数要求实现的功能是检查指定的信号量是否可用，这里调用 strcmp 函数判断信号量名称中是否存在“SEM”字段，如果存在说明是信号量，否则不是信号量。

sys_sem_set_invalid 函数要求实现的功能是使指定的信号量不可用，这里先调用

sys_sem_valid 确保信号量可用，再调用 sys_sem_free 函数释放该信号量。

代码清单 43-34 系统初始化

```
1 void sys_init(void)  
2 {  
3     OS_ERR ucErr;  
4     memset(LwIP_task_priority_stask, 0, sizeof(LwIP_task_priority_stask));  
5     /* init mem used by sys_mbox_t, use ucosIII functions */  
6     OSMemCreate(&StackMem, "LWIP TASK STK", (void*)LwIP_Task_Stk,  
7                 LWIP_TASK_MAX, LWIP_STK_SIZE*sizeof(CPU_STK), &ucErr);  
8     LWIP_ASSERT("sys_init: failed OSMemCreate STK", ucErr == OS_ERR_NONE );  
9 }
```

sys_init 函数在系统启动时被调用，可以用于初始化工作环境，这里先调用 memset 函数将 LwIP_task_priority_stask 数组内容清空，该数值用于存放任务优先级，接下来调用 OSMemCreate 函数初始化申请内存空间，用于 LwIP 任务栈空间。

代码清单 43-35 任务创建

```
1 sys_thread_t sys_thread_new(const char *name, lwip_thread_fn thread ,  
2                               void *arg, int stacksize, int prio)  
3 {  
4     CPU_INT08U ubPrio = LWIP_TASK_START_PRIO;  
5     OS_ERR ucErr;  
6     int i;  
7     int tsk_prio;  
8     CPU_STK * task_stk;  
9     if (prio) {  
10         ubPrio +=(prio-1);
```

```
11     for (i=0; i<LWIP_TASK_MAX; ++i)
12         if (LwIP_task_priority_stask[i] == ubPrio)
13             break;
14     if (i == LWIP_TASK_MAX) {
15         for (i=0; i<LWIP_TASK_MAX; ++i)
16             if (LwIP_task_priority_stask[i]==0) {
17                 LwIP_task_priority_stask[i] = ubPrio;
18                 break;
19             }
20         if (i == LWIP_TASK_MAX) {
21             LWIP_ASSERT("sys_thread_new: there is no space for priority",0);
22             return (-1);
23         }
24     } else
25         prio = 0;
26 }
27 /* Search for a suitable priority */
28 if (!prio) {
29     ubPrio = LWIP_TASK_START_PRIO;
30     while (ubPrio < (LWIP_TASK_START_PRIO+LWIP_TASK_MAX)) {
31         for (i=0; i<LWIP_TASK_MAX; ++i)
32             if (LwIP_task_priority_stask[i] == ubPrio) {
33                 ++ubPrio;
34                 break;
35             }
36         if (i == LWIP_TASK_MAX)
37             break;
38     }
39     if (ubPrio < (LWIP_TASK_START_PRIO+LWIP_TASK_MAX))
40         for (i=0; i<LWIP_TASK_MAX; ++i)
41             if (LwIP_task_priority_stask[i]==0) {
42                 LwIP_task_priority_stask[i] = ubPrio;
43                 break;
44             }
45     if (ubPrio >= (LWIP_TASK_START_PRIO+LWIP_TASK_MAX) || i == LWIP_TASK_MAX) {
46         LWIP_ASSERT("sys_thread_new: there is no free priority", 0);
47         return (-1);
48     }
49 }
50 if (stacksize > LWIP_STK_SIZE || !stacksize)
51     stacksize = LWIP_STK_SIZE;
52 /* get Stack from pool */
53 task_stk = OSMemGet( &StackMem, &ucErr );
54 if (ucErr != OS_ERR_NONE) {
55     LWIP_ASSERT("sys_thread_new: impossible to get a stack", 0);
56     return (-1);
57 }
58 tsk_prio = ubPrio-LWIP_TASK_START_PRIO;
59 OSTaskCreate(&LwIP_task_TCB[tsk_prio],
60             (CPU_CHAR *)name,
61             (OS_TASK_PTR)thread,
62             (void *)0,
63             (OS_PRIO )ubPrio,
64             (CPU_STK *)&task_stk[0],
65             (CPU_STK_SIZE)stacksize/10,
66             (CPU_STK_SIZE)stacksize,
67             (OS_MSG_QTY)0,
68             (OS_TICK )0,
69             (void *)0,
70             (OS_OPT ) (OS_OPT_TASK_STK_CHK | OS_OPT_TASK_STK_CLR),
71             (OS_ERR *)&ucErr);
72
73     return ubPrio;
74 }
```

`sys_thread_new` 函数要求实现的功能是新建一个任务，函数有五个形参，分别指定任务名称、任何函数、任务自定义参数、任务栈空间大小、任务优先级。对于 LwIP，系统有

限制其最多可用任务数，对于优先级也指定一定的范围，sys_thread_new 函数先对优先级参数进行处理，获取合适的优先级。OSMemGet 函数用于从分配给 LwIP 任务栈使用的内存空间中申请一块空间用于本任务。OSTaskCreate 函数用于创建一个任务。

代码清单 43-36 临界区域保护

```
1 sys_prot_t sys_arch_protect(void)
2 {
3     CPU_SR_ALLOC();
4
5     CPU_CRITICAL_ENTER();
6     return 1;
7 }
8
9 void sys_arch_unprotect(sys_prot_t pval)
10 {
11     CPU_SR_ALLOC();
12
13     LWIP_UNUSED_ARG(pval);
14     CPU_CRITICAL_EXIT();
15 }
```

sys_arch_protecth 函数要求实现的功能是完成临界区域保护并保存当前内容，这里调用 CPU_CRITICAL_ENTER 函数进入临界区域保护。sys_arch_unprotect 函数要求实现的功能是恢复受保护区域的先前状态，与 sys_arch_protecth 函数配套使用，这里直接调用 CPU_CRITICAL_EXIT 函数完成退出临界区域保护。

ethernetif.c 文件存放 LwIP 与 ETH 外设连接函数(网络接口函数)，属于最底层驱动函数，与上个实验无操作系统移植的文件内容有所不同，这里在函数内部会调用相关系统操作函数。

代码清单 43-37 low_level_init 函数

```
1 static void low_level_init(struct netif *netif)
2 {
3     uint32_t i;
4
5     /* set netif MAC hardware address length */
6     netif->hwaddr_len = ETHARP_HWADDR_LEN;
7     /* set netif MAC hardware address */
8     netif->hwaddr[0] = MAC_ADDR0;
9     netif->hwaddr[1] = MAC_ADDR1;
10    netif->hwaddr[2] = MAC_ADDR2;
11    netif->hwaddr[3] = MAC_ADDR3;
12    netif->hwaddr[4] = MAC_ADDR4;
13    netif->hwaddr[5] = MAC_ADDR5;
14    /* set netif maximum transfer unit */
15    netif->mtu = 1500;
16    /* Accept broadcast address and ARP traffic */
17    netif->flags=NETIF_FLAG_BROADCAST|NETIF_FLAG_ETHARP|NETIF_FLAG_LINK_UP;
18    s_pNetIf =netif;
19
20    /* initialize MAC address in ethernet MAC */
21    ETH_MACAddressConfig(ETH_MAC_Address0, netif->hwaddr);
22    /* Initialize Tx Descriptors list: Chain Mode */
23    ETH_DMATxDscrChainInit(DMATxDscrTab, &Tx_Buff[0][0], ETH_TXBUFNB);
24    /* Initialize Rx Descriptors list: Chain Mode */
25    ETH_DMARxDscrChainInit(DMARxDscrTab, &Rx_Buff[0][0], ETH_RXBUFNB);
26
27    /* Enable Ethernet Rx interrupt */
28    for (i=0; i<ETH_RXBUFNB; i++) {
29        ETH_DMARxDscrReceiveITConfig(&DMARxDscrTab[i], ENABLE);
30    }
31 }
```

```
32 #ifndef CHECKSUM_BY_HARDWARE
33     /* Enable the checksum insertion for the Tx frames */
34     {
35         for (i=0; i<ETH_TXBUFNB; i++) {
36             ETH_DMATxDescChecksumInsertionConfig(&DMATxDscrTab[i],
37             ETH_DMATxDesc_ChecksumTCPUDPICMPFull);
38         }
39     }
40 #endif
41
42     /* create the task that handles the ETH_MAC */
43     sys_thread_new((const char*)"Eth_if", ethernetif_input, netif,
44     netifINTERFACE_TASK_STACK_SIZE, netifINTERFACE_TASK_PRIORITY);
45     /* Enable MAC and DMA transmission and reception */
46     ETH_Start();
47 }
```

low_level_init 函数是网络接口初始化函数，在 ethernetif_init 函数被调用，在系统启动时被运行一次，用于初始化与网络接口相关硬件。函数先是给 netif 结构体成员赋值配置网卡参数，调用 ETH_MACAddressConfig 函数绑定网卡 MAC 地址，

ETH_DMATxDescChainInit 和 ETH_DMARxDescChainInit 初始化网络数据帧发送和接收描述符，设置为链模式。调用 ETH_DMARxDescReceiveITConfig 函数使能 DMA 数据接收相关中断。通过定义宏 CHECKSUM_BY_HARDWARE，可以使能发送数据硬件校验和，这个需要硬件支持，STM32F4xx 控制器是支持的。调用 sys_thread_new 函数创建一个任务，设置任务函数是 ethernetif_input，该函数用于讲接收到数据包转入到 LwIP 内部缓存区，这里还传递了 netif 结构体变量。最后，调用 ETH_Start 函数使能 ETH。

low_level_output 和 low_level_input 两个函数内容与上个实验工程同名函数几乎相同，这里不再讲解。

代码清单 43-38 ethernetif_input 函数

```
1 void ethernetif_input( void * pvParameters )
2 {
3     struct pbuf *p;
4     OS_ERR os_err;
5     err_t err;
6
7     /* move received packet into a new pbuf */
8     while (1) {
9         SYS_ARCH_DECL_PROTECT(sr);
10
11         SYS_ARCH_PROTECT(sr);
12         p = low_level_input(s_pxNetIf);
13         SYS_ARCH_UNPROTECT(sr);
14         if (p == NULL)
15             continue;
16         err = s_pxNetIf->input(p, s_pxNetIf);
17         if (err != ERR_OK) {
18             LWIP_DEBUGF(NETIF_DEBUG, ("ethernetif_input: IP input error\n"));
19             pbuf_free(p);
20             p = NULL;
21         }
22         /*sleep 5 ms*/
23         OSTimeDlyHMSM(0, 0, 0, 5, OS_OPT_TIME_DLY, (OS_ERR *)&os_err);
24     }
25 }
```

ethernetif_input 函数作为 sys_thread_new 指定的任务函数，用于接收网络数据包并将其转入到 LwIP 内核。SYS_ARCH_DECL_PROTECT、SYS_ARCH_PROTECT 和 SYS_ARCH_UNPROTECT 三个函数是与临界区域保护相关代码，可用在 lwipopts.h 文件中

的 SYS_LIGHTWEIGHT_PROT 配置相关功能。调用 low_level_input 函数获取接收到的数据包，如果判断没有接收到数据包则不执行本来循环后面内容。在判断接收到数据包后，将数据包内容转入 LwIP 内核。

相对与上个实验工程，那个时候我们需要在 main 函数中的无限循环中调用数据包接收查询，现在在这里我们创建一个这个数据包接收任务，让它执行数据包接收并将数据转入到 LwIP 内核。

代码清单 43-39 ethernetif_init 函数

```
1 err_t ethernetif_init(struct netif *netif)
2 {
3     LWIP_ASSERT("netif != NULL", (netif != NULL));
4
5     #if LWIP_NETIF_HOSTNAME
6         /* Initialize interface hostname */
7         netif->hostname = "lwip";
8     #endif /* LWIP_NETIF_HOSTNAME */
9
10    netif->name[0] = IFNAME0;
11    netif->name[1] = IFNAME1;
12
13    netif->output = etharp_output;
14    netif->linkoutput = low_level_output;
15
16
17    /* initialize the hardware */
18    low_level_init(netif);
19
20    etharp_init();
21    sys_timeout(ARP_TMR_INTERVAL, arp_timer, NULL);
22
23    return ERR_OK;
24 }
25
```

ethernetif_init 函数用于初始化网卡，在系统启动时必须被运行一次，该函数在 LwIP_Init 函数中被调用。函数首先为 netif 结构体成员赋值，然后调用 low_level_init 函数完成 ETH 外设初始化，etharp_init 函数完成 ARP 协议初始化，sys_timeout 函数启动 ARP 超时并注册一个 ARP 超时回调函数。

netconf.c 和 netconf.h 用于存放 LwIP 配置相关代码。netconf.h 是相关的宏定义，具体参考代码清单 43-8，不过本实验定义了 USE_DHCP 宏，使能 DHCP 功能。不同于上个实验，现在 netconf.c 文件只要求实现两个函数，LwIP_Init 和 LwIP_DHCP_task 函数，把其他函数删除。其中 LwIP_Init 函数与上个实验工程使用相同配置即可，参考代码清单 43-9。

代码清单 43-40 LwIP_DHCP_task 函数

```
1 #ifdef USE_DHCP
2 void LwIP_DHCP_task(void * pvParameters)
3 {
4     struct ip_addr ipaddr;
5     struct ip_addr netmask;
6     struct ip_addr gw;
7     OS_ERR os_err;
8
9     while (1) {
10         switch (DHCP_state) {
11             /* *****
12             /* 与上个实验工程代码相同，参考代码清单 43-12
13             /* *****
14         }
15     }
16 }
```

```
15         OSTimeDlyHMSM( 0u, 0u, 0u, 250u, OS_OPT_TIME_HMSM_STRICT, &os_err);
16     }
17 }
18 #endif
```

在 netconf.h 文件中定义了 USE_DHCP 宏，即开启了 DHCP 功能，LwIP_DHCP_task 函数才有效。在上个实验工程中，我们使用 LwIP_DHCP_Process_Handle 函数(参考代码清单 43-12)完成 DHCP 功能实现，该函数是被周期调用执行的。现在，既然我们使用了操作系统，就可以直接创建一个任务执行 DHCP 功能，LwIP_DHCP_task 函数就是 DHCP 任务函数，函数需要实现的内容与代码清单 43-12 相同，在函数最后调用 OSTimeDlyHMSM 函数延时 250ms。

lwipopts.h 文件存放一些宏定义，用于剪切 LwIP 功能，该文件拷贝自 ST 官方带操作系统的工程文件，方便我们移植，该文件同时使能了 Netconn 和 Socket 编程支持。这里我们还需要对该文件一个宏定义进行修改，直接把 TCPIP_THREAD_PRIO 宏定义为 6，该宏定义了 TCPIP 任务的优先级。

至此，有关 LwIP 函数文件修改已经全部完成，接下来还需要实现就是调用相关初始化函数完成 LwIP 初始化，然后就可以直接使用 LwIP 函数完成用户任务。

首先是 ETH 外设硬件相关初始化函数调用，bsp.c 文件中 BSP_Init 函数用于放置系统启动时各模块硬件初始化函数。

代码清单 43-41 BSP_Init 函数

```
1 void BSP_Init (void)
2 {
3     /* 设置 NVIC 优先级分组为 Group2: 0-3 抢占式优先级, 0-3 的响应式优先级 */
4     NVIC_PriorityGroupConfig(NVIC_PriorityGroup_2);
5
6     /* 初始化 LED */
7     LED_GPIO_Config();
8
9     Key_GPIO_Config();
10    /* 初始化调试串口, 一般为串口 1 */
11    Debug_USART_Config();
12
13    printf("基于 uCOS-III 的 LwIP 网络通信测试\n");
14
15    BSP_Tick_Init();
16
17    /* Configure ethernet (GPIOs, clocks, MAC, DMA) */
18    ETH_BSP_Config();
19    printf("LAN8720A BSP INIT AND COMFIGURE SUCCESS\n");
20 }
```

在 BSP_Init 函数最后调用 ETH_BSP_Config 函数完成 ETH 外设相关硬件初始化，包含了 RMII 和 SMI 相关 GPIO 初始化，ETH 外设时钟使能、MAC 和 DMA 配置并获取 PHY 的状态。

代码清单 43-42 开始任务函数

```
1 static void AppTaskStart (void *p_arg)
2 {
3     OS_ERR      err;
4     (void)p_arg;
5
6     BSP_Init();           /* Initialize BSP functions*/
7     CPU_Init();          /* Initialize the uC/CPU services*/
8
9     #if OS_CFG_STAT_TASK_EN > 0u
```

```
10     OSStatTaskCPUUsageInit(&err); /*Compute CPU capacity with no task running*/
11 #endif
12
13 #ifdef CPU_CFG_INT_DIS_MEAS_EN
14     CPU_IntDisMeasMaxCurReset();
15 #endif
16
17 #if (APP_CFG_SERIAL_EN == DEF_ENABLED)
18     APP_TRACE_DBG(("Creating Application kernel objects\n\r"));
19 #endif
20     AppObjCreate();
21     /* Create Appliaiton kernel objects */
22 #if (APP_CFG_SERIAL_EN == DEF_ENABLED)
23     APP_TRACE_DBG(("Creating Application Tasks\n\r"));
24 #endif
25     AppTaskCreate(); /* Create Application tasks*/
26
27     /* Initilaize the LwIP stack */
28     LwIP_Init();
29
30 #ifdef USE_DHCP
31     /* Start DHCPClient */
32     OSTaskCreate(&AppTaskDHCPSTCB, "DHCP",
33                 LwIP_DHCP_task,
34                 &gnetif,
35                 APP_CFG_TASK_DHCP_PRIO,
36                 &AppTaskDHCPStk[0],
37                 AppTaskDHCPStk[APP_CFG_TASK_DHCP_STK_SIZE / 10u],
38                 APP_CFG_TASK_DHCP_STK_SIZE,
39                 0u,
40                 0u,
41                 0,
42                 (OS_OPT_TASK_STK_CHK | OS_OPT_TASK_STK_CLR),
43                 &err);
44 #endif // #ifdef USE_DHCP
45
46     while (DEF_TRUE) {
47         OSTimeDlyHMSM(0u, 0u, 1u, 0u,
48                     OS_OPT_TIME_HMSM_STRICT,
49                     &err);
50     }
51 }
```

AppTaskStart 函数是系统运行的启动任务函数，先执行 BSP_Init 函数完成各个模块硬件初始化，接下来几个函数用于 uCOS-III 初始化，接下来调用 LwIP_Init 函数完成 LwIP 协议栈初始化。如果使能了 DHCP 功能，就创建 DHCP 任务，指定 LwIP_DHCP_task 函数为 DHCP 任务函数。

这个实验只是简单实现 LwIP 在 uCOS-III 操作系统基础上移植，并没有过多实现应用层方面代码，最后通过开发板是否 ping 通检验。

下载验证

保证开发板相关硬件连接正确，用 USB 线连接开发板“USB TO UART”接口跟电脑，在电脑端打开串口调试助手并配置好相关参数；使用网线连接开发板网口跟路由器，这里要求电脑连接在同一个路由器上，这里要求使用路由器，可以提供 DHCP 服务器功能，而电脑不行的。编译工程文件下载到开发板上。在串口调试助手可以看到相关信息，参考图 43-25，可以看到在使能 DHCP 功能之后，开发板动态获取 IP 地址为：192.168.1.124，这与我们在 netconf.h 文件中设置的静态地址是不同的(当然，也有可能刚好相同)。

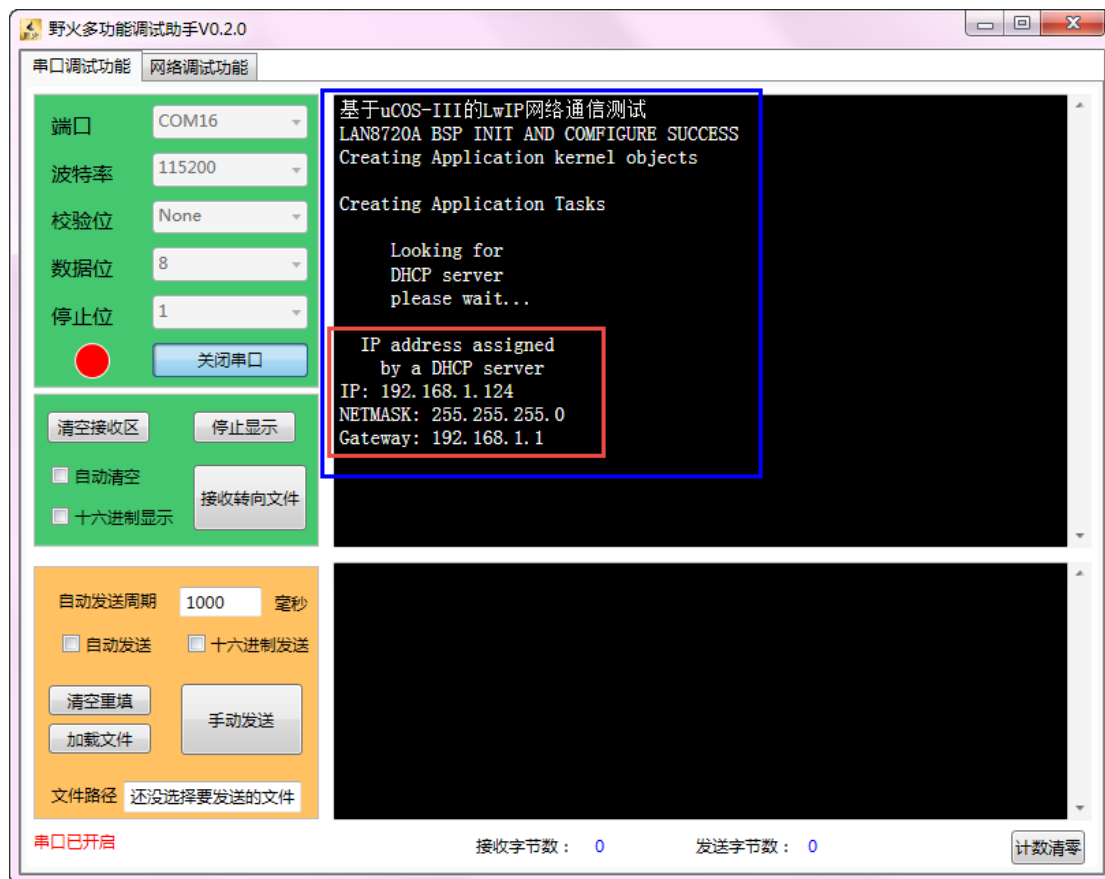


图 43-25 串口调试助手窗口

串口调试助手显示如图 43-25 信息是代码移植成功的最基本保证。如果没有代码没有移植成功或者网线没有接好，是无法通过 DHCP 获取动态 IP 的。保证移植成功之后，为进一步验证程序，我们可以在电脑端 ping 开发板网络。打开电脑端的 DOC 命令输入窗口，输入“ping 192.168.1.124”，就可以测试网络链路，链路正常时 DOC 窗口截图如图 43-26。

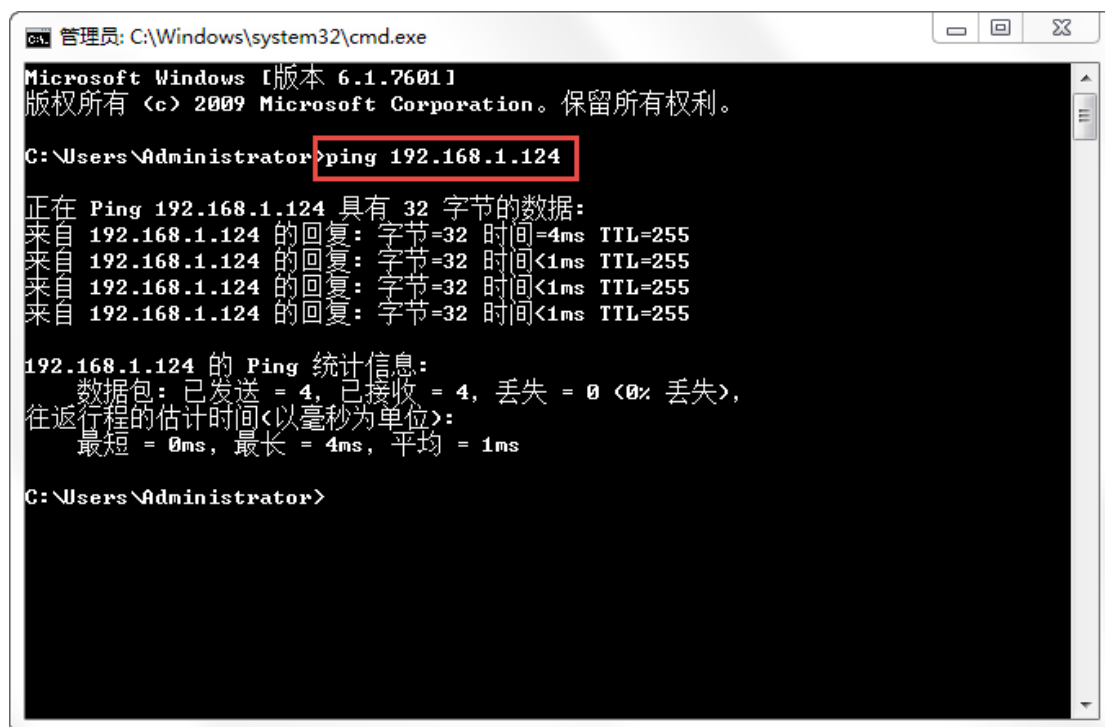


图 43-26 ping 命令窗口